KUET_Phantoms Team Notebook

Sakiful Siam, Nihan Mahmud,
Mohammad Adib Chowdhury

Contents

1 Data Structure 1
1.1 2D Fenwick Tree [19 lines] - B5B7C5C9 1
1.2 Coordinate Compression [10 lines] - 964F1135 . 1
1.3 DSU with Rollback [23 lines] - B9BE1F42 . . . 2
1.4 DSU [12 lines] - CBA010E3 2
1.5 Dynamic Segment Tree [32 lines] - 5E509C15 . 2
1.6 Fenwick Tree [16 lines] - 0A127E9F 2
1.7 Lazy Segment Tree [33 lines] - 8852D1B5 2
1.8 Li Chao Tree (Dynamic CHT) [23 lines] - 8A23162A 2
1.9 MO with Updates [22 lines] - DEEB577D 3
1.10 MO [21 lines] - 365FA0CD 3
1.11 Persistent Segment Tree [26 lines] - AD80C226 3
1.12 Segment Tree [26 lines] - 83059A84 3
1.13 Sieve [19 lines] - 33202376 3
1.14 Sqrt Decomposition [16 lines] - 3BE3519B . . . 3
1.15 Sqrt Tricks [15 lines] - 88D92062 4
1.16 Static Range Queries [33 lines] - 5B167B1B . . 4
1.17 Trie Bit [32 lines] - 02498358 4
1.18 Wavelet Tree [21 lines] - 13646CA5 4

2 Dynamic Programming 4
2.1 Bitmask DP [14 lines] - C38E4F22 4
2.2 CHT DP [21 lines] - 404403AC 4
2.3 DP Fundamentals [30 lines] - 01D2F8BF 5
2.4 DP with Fenwick Tree [16 lines] - 4EF7EF1F . . 5
2.5 DP with Segment Tree [24 lines] - 16E3C5F7 . 5
2.6 Digit DP [15 lines] - 719EC5D4 5
2.7 Expected Value DP [22 lines] - 52293ADF . . . 5
2.8 Knuth Optimization [19 lines] - E0D31F7B . . . 6
2.9 LIS and LCS [14 lines] - BE71A9A7 6
2.10 Matrix Exponentiation [20 lines] - 92FB502E . 6
2.11 SOS DP [11 lines] - 99BAFF2A 6

3 Flow 6
3.1 Dinic [72 lines] - A39F4E49 6
3.2 Hopcroft-Karp [44 lines] - 13DFB1D8 6
3.3 Min-Cost Max-Flow [38 lines] - 65176019 7

4 Game Theory 7
4.1 Points to be noted [14 lines] - 2A4DE83E 7

5 Geometry 7
5.1 Advanced Geometry Notes [28 lines] - 18AED942 7
5.2 Basic Geometry [20 lines] - D321C455 7
5.3 Convex Hull [14 lines] - ED89B1F4 8
5.4 Rotating Calipers [18 lines] - 4FFB9253 8

6 Graph 8
6.1 2-SAT [33 lines] - 817CF66C 8
6.2 Articulation Points and Bridges [26 lines] - 0579D87B 8
6.3 Bellman-Ford [17 lines] - 682245B2 8
6.4 Biconnected Components [36 lines] - 579687C0 8
6.5 Centroid Decomposition [11 lines] - 9BFB863B 9
6.6 DSU on Tree [19 lines] - 17DCA7F1 9
6.7 Eulerian Path and Circuit [32 lines] - F1583A3D 9
6.8 Floyd-Warshall [12 lines] - 3A4CAC57 9
6.9 Graph Basics [69 lines] - 54305FC5 9
6.10 Heavy Light Decomposition [18 lines] - A3AD48C0 10
6.11 Mo on Tree [25 lines] - 813F811E 10
6.12 Multi-source BFS [15 lines] - CF1E4F90 10
6.13 SCC [43 lines] - 498BB931 10
6.14 Tree DP [18 lines] - 3557D8FE 11
6.15 Tree Queries [28 lines] - 738F3996 11
6.16 kuhn [20 lines] - 3D9B43DE 11

7 Math 11
7.1 CRT [17 lines] - 6647AF89 11
7.2 Combinatorics and Probability [34 lines] - 103E7315 11
7.3 Euler Phi and Mobius [24 lines] - 4089BFF8 . . 12
7.4 Euler Theorem and Power Towers [16 lines] - 06028EF3 12
7.5 Exact GCD Counting [16 lines] - F3CAC16C . . 12
7.6 Linear Diophantine Equation [15 lines] - 8747D6A7 12
7.7 Lucas Theorem [21 lines] - 937ED62C 12
7.8 Miller-Rabin Pollard-Rho [43 lines] - 0FC59EF1 12
7.9 Modular Inverse [13 lines] - 6D6C77FA 12
7.10 Probability and Expected Value [13 lines] - 9669F842 13
7.11 Stirling Numbers [7 lines] - D3007645 13
7.12 Sum of Divisors [12 lines] - FCF9A6F7 13
7.13 XOR Hashing [15 lines] - 5F1277A0 13

8 Misc 13
8.1 Bit hacks [12 lines] - 1398A0E8 13
8.2 STL Reference [28 lines] - 14293C11 13
8.3 Template [17 lines] - 322F3F47 13

9 Searching 13
9.1 Basic Array Patterns [38 lines] - 1DE99FBA . . 13
9.2 Basic Binary Search [27 lines] - 2333BA5B . . . 14
9.3 Range and Search Techniques [41 lines] - 8926C3DB 14

10 String 14
10.1 Aho-Corasick [27 lines] - A686271F 14
10.2 KMP [17 lines] - A1049B24 14
10.3 Manacher [15 lines] - FBB15D9E 14
10.4 Rolling Hash [33 lines] - 87D05FA9 15
10.5 Suffix Array [29 lines] - 7C2108AD 15
10.6 Trie [15 lines] - 48982540 15
10.7 Z-Algorithm [10 lines] - 7FEC0BF7 15

11 Basic Math and Formula Sheet 15
11.1 Integer Arithmetic 15
11.2 Sums and Counting 15
11.3 Modular Arithmetic 15
11.4 Binary Search Checklist 15
11.5 Bit and Geometry Reminders 16
11.6 Combinatorics 16
11.6.1 Catalan Number 16
11.6.2 Stirling Numbers of the Second Kind 16
11.6.3 Lucas Theorem 16
11.6.4 Derangement 16
11.7 Number Theory 16
11.7.1 Mobius Function and Inversion 16
11.7.2 GCD and LCM 16
11.7.3 Pick's Theorem 17
11.7.4 Formula Cheatsheet 17

1 Data Structure

1.1 2D Fenwick Tree [19 lines] - B5B7C5C9

```
// Point add and rectangle sum on a 1-indexed grid.
    TIME: O(log n log m).
struct Fenwick2D {
    int n, m;
    vector<vector<long long>> bit;
    Fenwick2D(int n, int m) : n(n), m(m), bit(n+1,
        vector<long long>(m+1)) {}
    void add(int x, int y, long long value) {
        for (int i=x; i<=n; i+=i&-i)
            for (int j=y; j<=m; j+=j&-j) bit[i][j] +=
                value;
    }
    long long sumPrefix(int x, int y) const {
        long long ans=0;
        for (int i=x; i>0; i-=i&-i)
            for (int j=y; j>0; j-=j&-j) ans +=
                bit[i][j];
        return ans;
    }
    long long rectangleSum(int x1,int y1,int x2,int y2)
        const {
        return sumPrefix(x2,y2)-sumPrefix(x1-1,y2)-
            sumPrefix(x2,y1-1)+sumPrefix(x1-1,y1-1);
    }
};
```

1.2 Coordinate Compression [10 lines] - 964F1135

```
// WHEN: replace large ordered values with compact
    ranks. TIME: build O(n log n), lookup O(log n).
    ASSUMES: add all values before build; id is
    meaningful for values present in the list.
// Coordinate compression: preserves order while mapping
    values to [0..m-1].
template<class T>
struct Compressor {
    vector<T> values;
    void add(T x){values.push_back(x);}
    void build(){sort(values.begin(),values.end());
        values.erase(unique(values.begin(),
        values.end()),values.end());}
    int id(const T& x) const {return
        lower_bound(values.begin(),values.end(),x)-
        values.begin();}
    T value(int i) const {return values[i];}
};
```

1.3 DSU with Rollback [23 lines] - B9BE1F42

```
// WHEN: undo unions in offline dynamic connectivity.
    TIME: find/unite O(log n), rollback O(changes).
    ASSUMES: no path compression; snapshot is history
    size.
// No path compression: union by size keeps tree height
    O(log n), enabling rollback.
struct RollbackDSU {
    vector<int> parent,size;
    vector<pair<int,int>> history; // {child root, old
        size of receiving root}; {-1,-1} = no-op
    int components;
    RollbackDSU(int n):parent(n),size(n,1),
        components(n){iota(parent.begin(),parent.end(),
        0);}
    int find(int v) const { while(v!=parent[v])
        v=parent[v]; return v; }
    int snapshot() const { return history.size(); }
    bool unite(int a,int b) {
        a=find(a); b=find(b);
        if(a==b) { history.push_back({-1,-1}); return
            false; }
        if(size[a]<size[b]) swap(a,b);
        history.push_back({b,size[a]}); parent[b]=a;
            size[a]+=size[b]; --components; return
            true;
    }
    void rollback(int snap) {
        while((int)history.size()>snap) {
            auto [b,oldSize]=history.back();
                history.pop_back();
            if(b==-1) continue;
            int a=parent[b]; parent[b]=b;
                size[a]=oldSize; ++components;
        }
    }
};
```

1.4 DSU [12 lines] - CBA010E3

```
struct DSU {
    vector<int> parent, size;
    DSU(int n) : parent(n), size(n, 1) {
        iota(parent.begin(), parent.end(), 0); }
    int find(int v) { return parent[v] == v ? v :
        parent[v] = find(parent[v]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a; size[a] += size[b];
        return true;
    }
};
```

1.5 Dynamic Segment Tree [32 lines] - 5E509C15

```
// WHEN: sparse range add/sum over a huge domain. TIME:
    O(log C) per operation. ASSUMES: coordinates in
    [0,1e9]; sums fit long long.
// Dynamic range add / range sum on a large fixed domain
    [0, LIMIT].
struct DynamicSegmentTree {
    static const int LIMIT=1000000000;
    struct Node { long long sum=0,lazy=0; int
        left=-1,right=-1; };
    vector<Node> t{Node()};
    int makeNode(){t.push_back(Node());return
        (int)t.size()-1;}
    void apply(int p,int l,int r,long long
        x){t[p].sum+=(r-l+1LL)*x;t[p].lazy+=x;}
    void push(int p,int l,int r){
        if(l==r||t[p].lazy==0)return;
        int m=l+(r-l)/2;
        if(t[p].left==-1)t[p].left=makeNode();
            if(t[p].right==-1)t[p].right=makeNode();
        int a=t[p].left,b=t[p].right; long long
            z=t[p].lazy; t[p].lazy=0;
        apply(a,l,m,z); apply(b,m+1,r,z);
    }
    void add(int ql,int qr,long long x,int p,int l,int
        r){
        if(qr<l||r<ql)return;
            if(ql<=l&&r<=qr){apply(p,l,r,x);return;}
        push(p,l,r); int m=l+(r-l)/2;
        if(t[p].left==-1)t[p].left=makeNode();
            if(t[p].right==-1)t[p].right=makeNode();
        int a=t[p].left,b=t[p].right;
            add(ql,qr,x,a,l,m); add(ql,qr,x,b,m+1,r);
            t[p].sum=t[a].sum+t[b].sum;
    }
    long long sum(int ql,int qr,int p,int l,int r){
        if(qr<l||r<ql)return 0; if(ql<=l&&r<=qr)return
            t[p].sum;
        if(t[p].left==-1&&t[p].right==-1)return
            t[p].lazy*(min(r,qr)-max(l,ql)+1LL);
        push(p,l,r); int m=l+(r-l)/2; long long ans=0;
        if(t[p].left!=-1)ans+=sum(ql,qr,t[p].left,l,m);
        if(t[p].right!=-1)ans+=sum(ql,qr,t[p].right,m+
            1,r);
        return ans;
    }
    void add(int l,int r,long long
        x){add(l,r,x,0,0,LIMIT);}
    long long sum(int l,int r){return
        sum(l,r,0,0,LIMIT);}
};
```

1.6 Fenwick Tree [16 lines] - 0A127E9F

```
// 1-indexed; point add, prefix/range sum in O(log n).
template<class T>
struct Fenwick {
    int n;
    vector<T> bit;
    Fenwick(int n = 0) : n(n), bit(n + 1, T{}) {}
    void add(int i, T value) {
        for (; i <= n; i += i & -i) bit[i] += value;
    }
    T sumPrefix(int i) const {
        T ans{};
        for (; i > 0; i -= i & -i) ans += bit[i];
        return ans;
    }
    T sumRange(int l, int r) const { return sumPrefix(r)
        - sumPrefix(l - 1); }
};
```

1.7 Lazy Segment Tree [33 lines] - 8852D1B5

```
// Range add and range sum; 0-indexed inclusive ranges.
struct LazySegmentTree {
    int n;
    vector<long long> tree, lazy;
    LazySegmentTree(int n) : n(n), tree(4 * n), lazy(4
        * n) {}
    void apply(int p, int l, int r, long long value) {
        tree[p] += (r - l + 1LL) * value;
        lazy[p] += value;
    }
    void push(int p, int l, int r) {
        if (lazy[p] == 0 || l == r) return;
        int m = (l + r) / 2;
        apply(p * 2, l, m, lazy[p]); apply(p * 2 + 1, m
            + 1, r, lazy[p]);
        lazy[p] = 0;
    }
    void add(int ql, int qr, long long value, int p, int
        l, int r) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { apply(p, l, r,
            value); return; }
        push(p, l, r);
        int m = (l + r) / 2;
        add(ql, qr, value, p * 2, l, m); add(ql, qr,
            value, p * 2 + 1, m + 1, r);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }
    long long sum(int ql, int qr, int p, int l, int r) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[p];
        push(p, l, r);
        int m = (l + r) / 2;
        return sum(ql, qr, p * 2, l, m) + sum(ql, qr, p
            * 2 + 1, m + 1, r);
    }
    void add(int l, int r, long long value) { add(l, r,
        value, 1, 0, n - 1); }
    long long sum(int l, int r) { return sum(l, r, 1,
        0, n - 1); }
};
```

1.8 Li Chao Tree (Dynamic CHT) [23 lines] - 8A23162A

```
// WHEN: arbitrary line insertions and minimum queries.
    TIME: O(log C) per operation. ASSUMES: integer x in
    [-1e9,1e9]; products fit long long.
// Li Chao tree for minimum line queries on a fixed
    integer x-domain.
struct DynamicLiChao {
    static const int XMIN=-1000000000, XMAX=1000000000;
    struct Line { long long m=0,b=(1LL<<62); long long
        eval(int x) const { return m*x+b; } };
    struct Node { Line line; int left=-1,right=-1; };
    vector<Node> t{Node()};
    int makeNode(){t.push_back(Node());return
        (int)t.size()-1;}
    void add(Line nw,int p,int l,int r) {
        int m=l+(r-l)/2; bool
            lef=nw.eval(l)<t[p].line.eval(l),mid=
            nw.eval(m)<t[p].line.eval(m);
        if(mid)swap(nw,t[p].line); if(l==r)return;
        if(lef!=mid) {
            if(t[p].left==-1)t[p].left=makeNode(); int
            c=t[p].left; add(nw,c,l,m); }
        else { if(t[p].right==-1)t[p].right=makeNode();
            int c=t[p].right; add(nw,c,m+1,r); }
    }
    long long query(int x,int p,int l,int r) const {
        long long ans=t[p].line.eval(x); if(l==r)return
            ans; int m=l+(r-l)/2;
```

```
        int c=x<=m?t[p].left:t[p].right;
        if(c==-1)return ans;
        return min(ans,query(x,c,x<=m?l:m+1,x<=m?m:r));
    }
    void addLine(long long m,long long
        b){add({m,b},0,XMIN,XMAX);}
    long long query(int x) const {return
        query(x,0,XMIN,XMAX);}
};
```

1.9 MO with Updates [22 lines] - DEEB577D

```
// WHEN: offline range queries mixed with point
    assignments. TIME: about O((N+Q)N^(2/3)) moves.
    ASSUMES: 0-based inclusive ranges; each update
    records before/after values and each query its
    update-time.
// Offline range queries with point assignments. Example
    maintained answer: range sum.
struct MoUpdate { int pos, before, after; };
struct MoQuery3D { int l,r,time,id; };
vector<long long> moWithUpdates(vector<int> a,const
    vector<MoUpdate>& updates,vector<MoQuery3D> qs) {
    int n=a.size(),
        block=max(1,(int)pow(max(1,n),2.0/3));
    sort(qs.begin(),qs.end(),[&](auto x,auto y){
        int bx=x.l/block,by=y.l/block; if(bx!=by)return
            bx<by;
        int br=x.r/block,cr=y.r/block; if(br!=cr)return
            (bx&1)?br>cr:br<cr;
        return (br&1)?x.time>y.time:x.time<y.time;
    });
    vector<long long> ans(qs.size()); int L=0,R=-1,T=0;
        long long cur=0;
    auto add=[&](int i){cur+=a[i];}; auto remove=[&](int
        i){cur-=a[i];};
    auto apply=[&](int idx,bool forward){auto
        u=updates[idx]; int
        value=forward?u.after:u.before;
        if(L<=u.pos&&u.pos<=R) remove(u.pos);
            a[u.pos]=value; if(L<=u.pos&&u.pos<=R)
            add(u.pos);};
    for(auto q:qs) {
        while(T<q.time) apply(T++,true); while(T>q.time)
            apply(--T,false);
        while(L>q.l)add(--L); while(R<q.r)add(++R);
            while(L<q.l)remove(L++);
            while(R>q.r)remove(R--);
        ans[q.id]=cur;
    }
    return ans;
}
```

1.10 MO [21 lines] - 365FA0CD

```
// Static range queries. Implement add/remove to
    maintain the desired answer.
struct Query { int l, r, id; };
vector<long long> moQueries(const vector<int>& a,
    vector<Query> qs) {
    int n = a.size(), block = max(1, (int)sqrt(n));
    sort(qs.begin(), qs.end(), [&](const Query& x, const
        Query& y) {
        int bx = x.l / block, by = y.l / block;
        return bx == by ? ((bx & 1) ? x.r > y.r : x.r <
            y.r) : bx < by;
    });
    vector<long long> ans(qs.size());
    int L = 0, R = -1; long long cur = 0;
    auto add = [&](int i) { cur += a[i]; };
    auto remove = [&](int i) { cur -= a[i]; };
    for (auto q : qs) {
        while (L > q.l) add(--L);
        while (R < q.r) add(++R);
        while (L < q.l) remove(L++);
        while (R > q.r) remove(R--);
        ans[q.id] = cur;
    }
    return ans;
}
```

1.11 Persistent Segment Tree [26 lines] - AD80C226

```
// WHEN: retain prefix versions and answer range kth
    queries. TIME: O(log n) per insert/query. ASSUMES:
    nonempty input; values are coordinate-compressed.
// Persistent frequency tree over compressed values; kth
    smallest in a subarray.
struct PersistentSegmentTree {
    struct Node { int left=0,right=0,sum=0; };
    vector<Node> t{{}}; vector<int> root; int n;
    PersistentSegmentTree(const vector<int>& a) {
        vector<int> v=a; sort(v.begin(),v.end());
            v.erase(unique(v.begin(),v.end()),v.end());
            n=v.size();
        root.push_back(0);
        for(int x:a) root.push_back(insert(root.back(),
            0,n-1,lower_bound(v.begin(),v.end(),x)-
            v.begin()));
        values=move(v);
    }
    vector<int> values;
    int insert(int old,int l,int r,int pos) {
        int cur=t.size(); t.push_back(t[old]);
            ++t[cur].sum;
        if(l<r) { int m=(l+r)/2; if(pos<=m)t[cur].left=
            insert(t[old].left,l,m,pos); else
            t[cur].right=insert(t[old].right,m+1,r,
            pos); }
        return cur;
    }
    int kthIndex(int leftRoot,int rightRoot,int k,int
        l,int r) const {
        if(l==r)return l;
        int leftCount=t[t[rightRoot].left].sum-
            t[t[leftRoot].left].sum,m=(l+r)/2;
        return k<=leftCount?kthIndex(t[leftRoot].left,
            t[rightRoot].left,k,l,
            m):kthIndex(t[leftRoot].right,
            t[rightRoot].right,k-leftCount,m+1,r);
    }
    int kth(int l,int r,int k) const { // inclusive
        zero-based [l,r]
        return values[kthIndex(root[l],root[r+1],k,0,n-
            1)];
    }
};
```

1.12 Segment Tree [26 lines] - 83059A84

```
// 0-indexed inclusive ranges; point assignment and
    range sum in O(log n).
struct SegmentTree {
    int n;
    vector<long long> tree;
    SegmentTree(const vector<long long>& a) :
        n((int)a.size()), tree(4 * max(1,
        (int)a.size())) { build(1, 0, n - 1, a); }
    void build(int p, int l, int r, const vector<long
        long>& a) {
        if (l == r) { tree[p] = a[l]; return; }
        int m = (l + r) / 2;
        build(p * 2, l, m, a); build(p * 2 + 1, m + 1,
            r, a);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }
    void setPoint(int i, long long value, int p, int l,
        int r) {
        if (l == r) { tree[p] = value; return; }
        int m = (l + r) / 2;
        if (i <= m) setPoint(i, value, p * 2, l, m);
            else setPoint(i, value, p * 2 + 1, m + 1,
            r);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }
    long long query(int ql, int qr, int p, int l, int r)
        const {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[p];
        int m = (l + r) / 2;
        return query(ql, qr, p * 2, l, m) + query(ql,
            qr, p * 2 + 1, m + 1, r);
    }
    void setPoint(int i, long long value) { setPoint(i,
        value, 1, 0, n - 1); }
    long long query(int l, int r) const { return
        query(l, r, 1, 0, n - 1); }
};
```

1.13 Sieve [19 lines] - 33202376

```
// Sieve of Eratosthenes
// O(n log log n)
vector<bool> isPrime(n + 1, true);
isPrime[0] = isPrime[1] = false;
for(int i = 2; i * i <= n; i++)
    if(isPrime[i])
        for(int j = i * i; j <= n; j += i)
            isPrime[j] = false;


// Sieve + Smallest Prime Factor (SPF)
// O(n log log n)
vector<int> spf(n + 1);
for(int i = 0; i <= n; i++)
    spf[i] = i;
for(int i = 2; i * i <= n; i++)
    if(spf[i] == i)
        for(int j = i * i; j <= n; j += i)
            if(spf[j] == j)
                spf[j] = i;
```

1.14 Sqrt Decomposition [16 lines] - 3BE3519B

```
// Range sum with point updates, 0-indexed; O(sqrt(n))
    per operation.
struct SqrtSum {
    int n, blockSize;
    vector<long long> a, block;
    SqrtSum(vector<long long> values) :
        n(values.size()), blockSize(max(1,
        (int)sqrt(n))), a(move(values)), block((n +
        blockSize - 1) / blockSize) {
```

```
        for (int i = 0; i < n; ++i) block[i / blockSize]
            += a[i];
    }
    void setValue(int i, long long value) { block[i /
        blockSize] += value - a[i]; a[i] = value; }
    long long query(int l, int r) const {
        long long ans = 0;
        while (l <= r && l % blockSize) ans += a[l++];
        while (l + blockSize - 1 <= r) { ans += block[l
            / blockSize]; l += blockSize; }
        while (l <= r) ans += a[l++];
        return ans;
    }
};
```

1.15 Sqrt Tricks [15 lines] - 88D92062

```
// Enumerate divisors in pairs up to sqrt(n),
    O(sqrt(n)).
vector<long long> divisors(long long n) {
    vector<long long> small, large;
    for(long long d=1;d<=n/d;++d) if(n%d==0) {
        small.push_back(d);
        if(d!=n/d) large.push_back(n/d);
    }
    reverse(large.begin(),large.end());
    small.insert(small.end(),large.begin(),large.end());
    return small;
}
// Group equal floor(n/i) values: ranges with the same
    quotient can be processed together.
template<class F> void forEqualQuotients(long long n,F
    process) {
    for(long long l=1,r;l<=n;l=r+1) { long long q=n/l;
        r=n/q; process(l,r,q); }
}
```

1.16 Static Range Queries [33 lines] - 5B167B1B

```
// Sparse table: immutable array, O(1) idempotent
    queries (min/max/gcd).
struct SparseTable {
    vector<vector<int>> st;
    vector<int> lg;
    SparseTable(const vector<int>& a) {
        int n = a.size(); lg.resize(n + 1);
        for (int i = 2; i <= n; ++i) lg[i] = lg[i/2] +
            1;
        st.assign(lg[n] + 1, vector<int>(n)); st[0] = a;
        for (int k = 1; k < (int)st.size(); ++k)
            for (int i = 0; i + (1 << k) <= n; ++i)
                st[k][i] = min(st[k-1][i], st[k-1][i +
                    (1 << (k-1))]);
    }
    int queryMin(int l, int r) const {
        int k = lg[r-l+1]; return min(st[k][l],
            st[k][r-(1<<k)+1]);
    }
};

// Merge sort tree: static range count of values <= x,
    O(log^2 n) per query.
struct MergeSortTree {
    int n; vector<vector<int>> t;
    MergeSortTree(const vector<int>& a) : n(a.size()),
        t(4 * max(1, (int)a.size())) {
        build(1,0,n-1,a); }
    void build(int p,int l,int r,const vector<int>& a) {
        if (l==r) { t[p]={a[l]}; return; }
        int m=(l+r)/2; build(p*2,l,m,a);
            build(p*2+1,m+1,r,a);
        merge(t[p*2].begin(),t[p*2].end(),t[p*2+
            1].begin(),t[p*2+1].end(),
            back_inserter(t[p]));
    }
    int countLE(int ql,int qr,int x,int p,int l,int r)
        const {
        if (qr<l || r<ql) return 0;
        if (ql<=l && r<=qr) return
            upper_bound(t[p].begin(),t[p].end(),x)-
            t[p].begin();
        int m=(l+r)/2; return countLE(ql,qr,x,p*2,l,m)+
            countLE(ql,qr,x,p*2+1,m+1,r);
    }
    int countLE(int l,int r,int x) const { return
        countLE(l,r,x,1,0,n-1); }
};
```

1.17 Trie Bit [32 lines] - 02498358

```
// Binary trie for nonnegative 31-bit values; increase
    the top bit if constraints require it.
struct BinaryTrie {
    struct Node { int child[2] = {-1, -1}; };
    vector<Node> t{{}};
    void insert(int x) {
        int v = 0;
        for (int b = 30; b >= 0; --b) {
            int bit = (x >> b) & 1;
            if (t[v].child[bit] == -1) { t[v].child[bit]
                = t.size(); t.push_back({}); }
            v = t[v].child[bit];
        }
    }
    int maxXor(int x) const {
        int v = 0, result = 0;
        for (int b = 30; b >= 0; --b) {
            int bit = (x >> b) & 1, want = bit ^ 1;
            if (t[v].child[want] != -1) { result |= 1 <<
                b; v = t[v].child[want]; }
            else v = t[v].child[bit];
        }
        return result;
    }
};
// Max prefix-XOR versus suffix-XOR over every split
    (both sides nonempty).
int maxPrefixSuffixXor(const vector<int>& a) {
    int n=a.size(); if(n<2) return 0;
    vector<int> pref(n),suff(n); pref[0]=a[0];
    for(int i=1;i<n;++i) pref[i]=pref[i-1]^a[i];
    suff[n-1]=a[n-1]; for(int i=n-2;i>=0;--i)
        suff[i]=suff[i+1]^a[i];
    BinaryTrie trie; int answer=0;
    for(int split=0;split<n-1;++split) {
        trie.insert(pref[split]);
        answer=max(answer,trie.maxXor(suff[split+1])); }
    return answer;
}
```

1.18 Wavelet Tree [21 lines] - 13646CA5

```
// WHEN: static range kth queries. TIME: O(log sigma)
    per query. ASSUMES: values fit the constructor
    domain; array does not change.
// Static range kth-smallest. Build on values in
    [lo,hi]; query uses inclusive zero-based indices.
struct WaveletTree {
    int lo,hi; vector<int> leftCount;
        unique_ptr<WaveletTree> left,right;
    WaveletTree(const vector<int>& a,int x,int
        y):lo(x),hi(y) {
        leftCount.assign(a.size()+1,0);
            if(a.empty()||lo==hi)return;
        int mid=lo+(hi-lo)/2; vector<int> l,r;
            l.reserve(a.size()); r.reserve(a.size());
        for(int i=0;i<(int)a.size();++i) {
            bool goLeft=a[i]<=mid;
                leftCount[i+1]=leftCount[i]+goLeft;
            (goLeft?l:r).push_back(a[i]);
        }
        if(!l.empty())left=make_unique<WaveletTree>(l,
            lo,mid);
        if(!r.empty())right=make_unique<WaveletTree>(r,
            mid+1,hi);
    }
    int kth(int l,int r,int k) const { // k is
        1-indexed
        if(lo==hi)return lo;
        int inLeft=leftCount[r+1]-leftCount[l];
        if(k<=inLeft)return
            left->kth(leftCount[l],leftCount[r+1]-1,k);
        return right->kth(l-leftCount[l],r-leftCount[r+
            1],k-inLeft);
    }
};
```

2 Dynamic Programming

2.1 Bitmask DP [14 lines] - C38E4F22

```
// Assignment-style bitmask DP: n <= about 20. dp[mask]
    = best cost after choosing mask.
long long assignmentDP(const vector<vector<int>>& cost)
    {
    int n=cost.size(), N=1<<n; const long long
        INF=(1LL<<60);
    vector<long long> dp(N,INF); dp[0]=0;
    for(int mask=0;mask<N;++mask) {
        int row=__builtin_popcount((unsigned)mask);
        if(row==n || dp[mask]==INF) continue;
        for(int col=0;col<n;++col) if(!(mask>>col&1)) {
            int next=mask|(1<<col); dp[next]=
                min(dp[next],dp[mask]+cost[row][col]);
        }
    }
    return dp[N-1];
}
```

2.2 CHT DP [21 lines] - 404403AC

```
// WHEN: DP transitions are line queries with ordered
    slopes and x values. TIME: O(1) amortized per
    operation. ASSUMES: slopes decrease and query x
    increases.
// Monotone convex hull trick for maximum of m*x+b:
    slopes decreasing, x queries increasing.
struct MonotoneCHT {
    struct Line { long long m,b; long long at(long long
        x) const { return m*x+b; } };
    deque<Line> hull;
```

```
    bool redundant(Line a,Line b,Line c) {
        return (__int128)(b.b-a.b)*(b.m-c.m)>=
            (__int128)(c.b-b.b)*(a.m-b.m);
    }
    void add(long long m,long long b) {
        Line x{m,b}; if(!hull.empty()&&hull.back().m==m)
            {
            if(hull.back().b>=b)return; hull.pop_back();
        }
        while(hull.size()>=
            2&&redundant(hull[hull.size()-2],
            hull.back(),x))hull.pop_back();
        hull.push_back(x);
    }
    long long query(long long x) {
        while(hull.size()>=2&&hull[0].at(x)<=
            hull[1].at(x))hull.pop_front();
        return hull.front().at(x);
    }
};
// Example recurrence: dp[i] = max_j(m[j]*x[i]+b[j]);
    add each candidate line, then query x[i].
```

2.3 DP Fundamentals [30 lines] - 01D2F8BF

```
// State design: define what dp[...] means, transitions,
    base cases, and evaluation order.
// 0/1 knapsack: iterate capacity downward so each item
    is used at most once.
long long knapsack01(const vector<int>& w, const
    vector<long long>& val, int cap) {
    vector<long long> dp(cap + 1);
    for (int i = 0; i < (int)w.size(); ++i)
        for (int c = cap; c >= w[i]; --c) dp[c] =
            max(dp[c], dp[c-w[i]] + val[i]);
    return dp[cap];
}
// Unbounded coin change: iterate amount upward to allow
    repeated use.
long long minCoins(const vector<int>& coin, int amount)
    {
    const int INF = 1e9; vector<int> dp(amount+1, INF);
        dp[0]=0;
    for (int x=1;x<=amount;++x) for (int c:coin) if
        (c<=x) dp[x]=min(dp[x],dp[x-c]+1);
    return dp[amount] == INF ? -1 : dp[amount];
}
// Grid paths with obstacles: '#' blocked; reconstruct
    one shortest right/down path.
string gridPath(const vector<string>& g) {
    int n=g.size(), m=g[0].size(); vector<vector<int>>
        dp(n,vector<int>(m,1e9));
    if(g[0][0]=='#') return ""; dp[0][0]=0;
    for(int i=0;i<n;++i) for(int j=0;j<m;++j)
        if(g[i][j]!='#') {
        if(i) dp[i][j]=min(dp[i][j],dp[i-1][j]+1);
        if(j) dp[i][j]=min(dp[i][j],dp[i][j-1]+1);
    }
    if(dp[n-1][m-1]>=1e9) return "";
    string path; int i=n-1,j=m-1;
    while(i||j) { if(i && dp[i][j]==dp[i-1][j]+1)
        path+='D',--i; else path+='R',--j; }
    reverse(path.begin(),path.end()); return path;
}
// Real-valued DP: use double states and explicit
    epsilon only for comparisons if needed.
// State swapping / rolling array: retain only the
    previous row when transitions need it.
// Recursion-to-iteration: process states in dependency
    order; replace recursion stack with a loop.
```

2.4 DP with Fenwick Tree [16 lines] - 4EF7EF1F

```
// WHEN: DP needs prefix aggregates after coordinate
    compression. TIME: O(n log n) here. ASSUMES:
    strictly increasing subsequence; max-sum variant.
// Maximum-sum strictly increasing subsequence;
    coordinate compression + Fenwick max.
struct FenwickMax {
    int n; vector<long long> bit;
    FenwickMax(int n):n(n),bit(n+1,0){}
    void update(int i,long long x) { for(;i<=n;i+=i&-i)
        bit[i]=max(bit[i],x); }
    long long query(int i) const { long long ans=0;
        for(;i;i-=i&-i) ans=max(ans,bit[i]); return
        ans; }
};
long long maxIncreasingSubsequenceSum(const vector<int>&
    a) {
    if(a.empty()) return 0;
    vector<int> vals=a; sort(vals.begin(),vals.end());
        vals.erase(unique(vals.begin(),vals.end()),
        vals.end());
    FenwickMax fw(vals.size()); long long
        answer=LLONG_MIN;
    for(int x:a) { int r=lower_bound(vals.begin(),
        vals.end(),x)-vals.begin()+1;
        long long cur=max((long
            long)x,fw.query(r-1)+x); fw.update(r,cur);
            answer=max(answer,cur); }
    return answer;
}
```

2.5 DP with Segment Tree [24 lines] - 16E3C5F7

```
// WHEN: DP needs range aggregates over compressed
    states. TIME: O(n log n) here. ASSUMES: strictly
    increasing subsequence; max-sum variant.
// Maximum-sum strictly increasing subsequence, using a
    segment tree over compressed values.
struct MaxSegTree {
    int n; vector<long long> t;
    MaxSegTree(int n):n(n),t(4*max(1,n),0){}
    void chmax(int i,long long x,int p,int l,int r) {
        if(l==r) { t[p]=max(t[p],x); return; }
        int m=(l+r)/2; if(i<=m) chmax(i,x,p*2,l,m); else
            chmax(i,x,p*2+1,m+1,r);
        t[p]=max(t[p*2],t[p*2+1]);
    }
    long long query(int ql,int qr,int p,int l,int r)
        const {
        if(qr<l||r<ql) return 0; if(ql<=l&&r<=qr) return
            t[p];
        int m=(l+r)/2; return max(query(ql,qr,p*2,l,m),
            query(ql,qr,p*2+1,m+1,r));
    }
    void chmax(int i,long long x) { chmax(i,x,1,0,n-1);
        }
    long long query(int l,int r) const { return
        l>r?0:query(l,r,1,0,n-1); }
};
long long maxIncreasingSubsequenceSumSeg(const
    vector<int>& a) {
    if(a.empty()) return 0;
    vector<int> v=a; sort(v.begin(),v.end());
        v.erase(unique(v.begin(),v.end()),v.end());
    MaxSegTree st(v.size()); long long ans=LLONG_MIN;
    for(int x:a) { int
        i=lower_bound(v.begin(),v.end(),x)-v.begin();
        long long cur=max((long
        long)x,st.query(0,i-1)+x); st.chmax(i,cur);
        ans=max(ans,cur); }
    return ans;
}
```

2.6 Digit DP [15 lines] - 719EC5D4

```
// Count integers in [0, bound] whose digit sum is
    divisible by k.
long long countDigitSumDivisible(const string& bound,int
    k) {
    int n=bound.size();
    vector<vector<array<long long,2>>>
        memo(n+1,vector<array<long long,2>>(k));
    vector<vector<array<char,2>>>
        seen(n+1,vector<array<char,2>>(k));
    function<long long(int,int,int)> dfs=[&](int pos,int
        rem,int tight)->long long {
        if(pos==n) return rem==0;
        if(seen[pos][rem][tight]) return
            memo[pos][rem][tight];
        seen[pos][rem][tight]=1;
        int lim=tight?bound[pos]-'0':9; long long ans=0;
        for(int d=0;d<=lim;++d)
            ans+=dfs(pos+1,(rem+d)%k,tight&&(d==lim));
        return memo[pos][rem][tight]=ans;
    };
    return dfs(0,0,1);
}
```

2.7 Expected Value DP [22 lines] - 52293ADF

```
// Example state equations: expected coin tosses until
    two consecutive heads.
// State 0: previous toss is not H; state 1: previous
    toss is H; state 2 is terminal.
// E0=1+(E0+E1)/2 and E1=1+E0/2, so E0=6 and E1=4.
pair<double,double> expectedTossesForHH(){return
    {6.0,4.0};}
// For a finite acyclic process, fill E[state] from
    terminal states backward using
// E[s] = immediate_cost + sum(P(s->t) * E[t]). For
    cyclic transitions solve equations.

// Gaussian elimination for cyclic Markov expectation
    equations.
vector<double> solveLinearSystem(vector<vector<double>>
    a) { // augmented n x (n+1)
    int n=a.size();
    for(int col=0,row=0;col<n&&row<n;++col) {
        int pivot=row; for(int
            i=row+1;i<n;++i)if(abs(a[i][col])>
            abs(a[pivot][col]))pivot=i;
        if(abs(a[pivot][col])<1e-12)continue;
        swap(a[pivot],a[row]); double div=a[row][col];
        for(int j=col;j<=n;++j)a[row][j]/=div;
        for(int i=0;i<n;++i)if(i!=row){double
            factor=a[i][col];for(int
            j=col;j<=n;++j)a[i][j]-=factor*a[row][j];}
        ++row;
```

```
    }
    vector<double> x(n); for(int
        i=0;i<n;++i)x[i]=a[i][n]; return x;
}
// Build one equation per nonterminal state: E[u] -
    sum(P(u->v)*E[v]) = 1.
// Include only nonterminal v terms; terminal states
    have expectation 0.
```

2.8 Knuth Optimization [19 lines] - E0D31F7B

```
// WHEN: interval DP satisfies Knuth monotonicity. TIME:
    O(n^2). ASSUMES: prove the quadrangle
    inequality/monotone-opt property for the
    recurrence.
// Interval DP optimization. Requires a valid
    quadrangle/monotone-opt condition.
// Replace cost(l,r) with the problem's interval cost;
    indices are inclusive.
template<class Cost>
vector<vector<long long>> knuthDP(int n,Cost cost) {
    const long long INF=(1LL<<62);
    vector<vector<long long>> dp(n,vector<long
        long>(n));
    vector<vector<int>> opt(n,vector<int>(n));
    for(int i=0;i<n;++i)opt[i][i]=i;
    for(int len=2;len<=n;++len) for(int
        l=0;l+len<=n;++l) {
        int r=l+len-1; dp[l][r]=INF;
        int from=max(l,opt[l][r-1]),to=min(r-1,opt[l+
            1][r]);
        for(int k=from;k<=to;++k) {
            long long cur=(k>=l+1?dp[l][k]:0)+(k+1<=
                r?dp[k+1][r]:0)+cost(l,r);
            if(cur<dp[l][r])dp[l][r]=cur,opt[l][r]=k;
        }
    }
    return dp;
}
```

2.9 LIS and LCS [14 lines] - BE71A9A7

```
// LIS length in O(n log n), strictly increasing.
int lisLength(const vector<int>& a) {
    vector<int> tails;
    for (int x : a) { auto
        it=lower_bound(tails.begin(),tails.end(),x);
        if(it==tails.end()) tails.push_back(x); else
        *it=x; }
    return tails.size();
}
// LCS length in O(n*m); can reduce memory by swapping
    to keep the shorter string in columns.
int lcsLength(const string& a, const string& b) {
    vector<int> dp(b.size()+1);
    for(char x:a) { int diagonal=0; for(int
        j=1;j<=(int)b.size();++j) {
        int old=dp[j]; dp[j]=(x==b[j-1]?diagonal+
            1:max(dp[j],dp[j-1])); diagonal=old;
    }}
    return dp.back();
}
```

2.10 Matrix Exponentiation [20 lines] - 92FB502E

```
// Fibonacci in O(log n); adapt matrix and
    multiplication for a linear recurrence.
using Matrix = vector<vector<long long>>;
const long long MOD = 1000000007;
Matrix multiply(const Matrix& a,const Matrix& b) {
    int n=a.size(); Matrix c(n,vector<long long>(n));
    for(int i=0;i<n;++i) for(int k=0;k<n;++k) for(int
        j=0;j<n;++j)
        c[i][j]=(c[i][j]+a[i][k]*b[k][j])%MOD;
    return c;
}
Matrix power(Matrix a,long long e) {
    int n=a.size(); Matrix r(n,vector<long long>(n));
    for(int i=0;i<n;++i) r[i][i]=1;
    for(;e;e>>=1,a=multiply(a,a)) if(e&1)
        r=multiply(r,a);
    return r;
}
long long fibonacci(long long n) {
    if(n==0) return 0;
    Matrix base={{1,1},{1,0}};
    return power(base,n-1)[0][0];
}
```

2.11 SOS DP [11 lines] - 99BAFF2A

```
// WHEN: aggregate a function over all
    submasks/supermasks. TIME: O(k*2^k). ASSUMES: k is
    small enough for 2^k memory.
// Subset zeta transform: after this, f[mask] = sum of
    original f[sub] over sub subset of mask.
void subsetZeta(vector<long long>& f,int bits) {
    for(int b=0;b<bits;++b) for(int
        mask=0;mask<(1<<bits);++mask)
        if(mask>>b&1) f[mask]+=f[mask^(1<<b)];
}
// Superset zeta: f[mask] becomes sum of original f[sup]
    where sup contains mask.
void supersetZeta(vector<long long>& f,int bits) {
    for(int b=0;b<bits;++b) for(int
        mask=0;mask<(1<<bits);++mask)
        if(!(mask>>b&1)) f[mask]+=f[mask|(1<<b)];
}
```

3 Flow

3.1 Dinic [72 lines] - A39F4E49

```
/*.Complexity: O(V^2 E)
  .Call Dinic with total number of nodes.
  .Nodes start from 0.
  .Capacity is long long data.
  .make graph with create edge(u,v,capacity).
  .Get max flow with maxFlow(src,des).*/
#define eb emplace_back
struct Dinic {
  struct Edge {
    int u, v;
    ll cap, flow = 0;
    Edge() {}
    Edge(int u, int v, ll cap) :u(u), v(v), cap(cap) {}
  };
  int N;
  vector<Edge>edge;
  vector<vector<int>>adj;
  vector<int>d, pt;
  Dinic(int N) :N(N), edge(0), adj(N), d(N), pt(N) {}
  void addEdge(int u, int v, ll cap) {
    if (u == v) return;
    edge.eb(u, v, cap);
    adj[u].eb(edge.size() - 1);
    edge.eb(v, u, 0);
    adj[v].eb(edge.size() - 1);
  }
  bool bfs(int s, int t) {
    queue<int>q({ s });
    fill(d.begin(), d.end(), N + 1);
    d[s] = 0;
    while (!q.empty()) {
      int u = q.front();q.pop();
      if (u == t) break;
      for (int k : adj[u]) {
        Edge& e = edge[k];
        if (e.flow<e.cap && d[e.v]>d[e.u] + 1) {
          d[e.v] = d[e.u] + 1;
          q.emplace(e.v);
        }
      }
    }
    return d[t] != N + 1;
  }
  ll dfs(int u, int T, ll flow = -1) {
    if (u == T || flow == 0) return flow;
    for (int& i = pt[u];i < adj[u].size();i++) {
      Edge& e = edge[adj[u][i]];
      Edge& oe = edge[adj[u][i] ^ 1];
      if (d[e.v] == d[e.u] + 1) {
        ll amt = e.cap - e.flow;
        if (flow != -1 && amt > flow) amt = flow;
        if (ll pushed = dfs(e.v, T, amt)) {
          e.flow += pushed;
          oe.flow -= pushed;
          return pushed;
        }
      }
    }
    return 0;
  }
  ll maxFlow(int s, int t) {
    ll total = 0;
    while (bfs(s, t)) {
      fill(pt.begin(), pt.end(), 0);
      while (ll flow = dfs(s, t)) {
        total += flow;
      }
    }
    return total;
  }

};
```

3.2 Hopcroft-Karp [44 lines] - 13DFB1D8

```
// Maximum bipartite matching. Left vertices [0,n),
    right vertices [0,m).
// TIME: O(E sqrt(V)).
struct HopcroftKarp {
    int n, m;
    vector<vector<int>> g;
    vector<int> leftMatch, rightMatch, dist;
    int shortest;
    HopcroftKarp(int n, int m) : n(n), m(m), g(n),
        leftMatch(n,-1), rightMatch(m,-1), dist(n) {}
    void addEdge(int u, int v) { g[u].push_back(v); }
    bool bfs() {
        queue<int> q;
        for (int u=0; u<n; ++u) {
            if (leftMatch[u] == -1) dist[u]=0,
                q.push(u);
```

```
            else dist[u]=-1;
        }
        shortest=INT_MAX;
        while (!q.empty()) {
            int u=q.front(); q.pop();
            if (dist[u]+1>shortest) continue;
            for (int v:g[u]) {
                int w=rightMatch[v];
                if (w==-1) shortest=dist[u]+1;
                else if (dist[w]==-1)
                    dist[w]=dist[u]+1, q.push(w);
            }
        }
        return shortest!=INT_MAX;
    }
    bool dfs(int u) {
        for (int v:g[u]) {
            int w=rightMatch[v];
            if ((w==-1 && dist[u]+1==shortest) ||
                (w!=-1 && dist[w]==dist[u]+1 && dfs(w))
                    {
                leftMatch[u]=v; rightMatch[v]=u; return
                    true;
            }
        }
        dist[u]=-1; return false;
    }
    int maximumMatching() {
        int ans=0;
        while (bfs()) for (int u=0; u<n; ++u)
            if (leftMatch[u]==-1 && dfs(u)) ++ans;
        return ans;
    }
};
```

3.3 Min-Cost Max-Flow [38 lines] - 65176019

```
// Successive shortest augmenting paths with SPFA. TIME:
    O(F*V*E).
// Supports negative edge costs; assumes no reachable
    negative-cost cycle.
struct MinCostMaxFlow {
    struct Edge { int to, rev, cap; long long cost; };
    int n;
    vector<vector<Edge>> g;
    MinCostMaxFlow(int n) : n(n), g(n) {}
    void addEdge(int u,int v,int cap,long long cost) {
        Edge a{v,(int)g[v].size(),cap,cost},
            b{u,(int)g[u].size(),0,-cost};
        g[u].push_back(a); g[v].push_back(b);
    }
    pair<int,long long> run(int s,int t) {
        int flow=0; long long cost=0;
        const long long INF=(1LL<<60);
        while (true) {
            vector<long long> d(n,INF); vector<int>
                pv(n,-1), pe(n,-1);
            vector<char> inQ(n); queue<int> q; d[s]=0;
                q.push(s); inQ[s]=1;
            while (!q.empty()) {
                int v=q.front(); q.pop(); inQ[v]=0;
                for (int i=0;i<(int)g[v].size();++i) {
                    auto &e=g[v][i];
                    if (e.cap && d[e.to]>d[v]+e.cost) {
                        d[e.to]=d[v]+e.cost;
                            pv[e.to]=v; pe[e.to]=i;
                        if (!inQ[e.to]) q.push(e.to),
                            inQ[e.to]=1;
                    }
                }
            }
            if (d[t]==INF) break;
            int add=INT_MAX;
            for (int v=t;v!=s;v=pv[v])
                add=min(add,g[pv[v]][pe[v]].cap);
            for (int v=t;v!=s;v=pv[v]) {
                Edge &e=g[pv[v]][pe[v]]; e.cap-=add;
                    g[v][e.rev].cap+=add;
            }
            flow+=add; cost+=1LL*add*d[t];
        }
        return {flow,cost};
    }
};
```

4 Game Theory

4.1 Points to be noted [14 lines] - 2A4DE83E

```
>[First Write a Brute Force solution]
>Nim = all xor
>Misere Nim = Nim + corner case: if all piles are 1,
    reverse(nim)
>Bogus Nim = Nim
>Staircase Nim = Odd indexed pile Nim (Even indexed pile
    doesnt matter, as one player can give bogus moves to
    drop all even piles to ground)
>Sprague Grundy: [Every impartial game under the normal
    play convention is equivalent to a one-heap game of
    nim]
Every tree = one nim pile = tree root value; tree leaf
    value = 0; tree node value = mex of all child nodes.
[Careful: one tree node can become multiple new tree
    roots(multiple elements in one node), then the value
    of that node = xor of all those root values]
>Hackenbush(Given a rooted tree; cut an edge in one
    move; subtree under that edge gets removed; last
    player to cut wins):
 Colon:  //G(u) = (G(v1) + 1) ⊕ (G(v2) + 1) ⊕ ··· [v1, v2, ···
    are childs of u]
For multiple trees ans is their xor
>Hackenbush on graph (instead of tree given an rooted
    graph):
fusion: All edges in a cycle can be fused to get a tree
    structure; build a super node, connect some single
    nodes with that super node, number of single nodes
    is the number of edges in the cycle.
Sol: [Bridge component tree] mark all bridges, a group
    of edges that are not bridges, becomes one component
    and contributes number of edges to the hackenbush.
    (even number of edges contributes 0, odd number of
    edges contributes 1)
```

5 Geometry

5.1 Advanced Geometry Notes [28 lines] - 18AED942

```
CONVEX HULL
Use monotonic chain to keep only extreme points. Sort
    points lexicographically,
build lower and upper chains, and remove the last point
    while the turn is not
counterclockwise. Complexity O(n log n). The Convex
    Hull.cpp file contains code.

ROTATING CALIPERS
For a convex polygon in counterclockwise order, move a
    second pointer around
the polygon as the edge direction rotates. The area
    between an edge and the
opposite vertex increases until the antipodal vertex is
    reached. This gives
the farthest pair in O(n); see Rotating Calipers.cpp.
    Squared distances avoid
floating-point comparisons.

HALF-PLANE INTERSECTION
A directed line keeps the half-plane on its left. Sort
    lines by angle, remove
redundant parallel lines, and maintain a deque of
    candidate boundary lines.
When a new line arrives, remove the back/front line
    while their intersection
lies outside the new half-plane. Close the deque at the
    end. Typical complexity
O(n log n). Be explicit about parallel lines,
    empty/unbounded output, and
epsilon comparisons. For a bounded polygon, add a
    sufficiently large bounding
box before processing if the problem permits it.

MINKOWSKI SUM AND DIFFERENCE
For point sets A and B, A+B = {a+b : a in A, b in B};
    A-B = A+(-B).
For convex polygons, rotate each CCW polygon to its
    lowest (y,x) vertex, then
merge their cyclic edge vectors by polar angle.
    Equal-angle edges are combined.
This computes the result in O(n+m) after hulls are
    available. To test whether
convex polygons intersect, test whether the origin lies
    in A+(-B). Handle
degenerate point/segment polygons separately when the
    input allows them.
```

5.2 Basic Geometry [20 lines] - D321C455

```
// WHEN: orientation, segment intersection, and polygon
    area. TIME: O(1) per primitive, O(n) area. ASSUMES:
    integer coordinates and products fit long long.
// Basic integer geometry. Coordinates and cross
    products use long long.
struct Point { long long x,y; };
Point operator-(Point a,Point b){return
    {a.x-b.x,a.y-b.y};}
long long cross(Point a,Point b){return
    a.x*b.y-a.y*b.x;}
long long orient(Point a,Point b,Point c){return
    cross(b-a,c-a);}
bool onSegment(Point a,Point b,Point p){
    return orient(a,b,p)==0&&min(a.x,b.x)<=p.x&&p.x<=
        max(a.x,b.x)&&min(a.y,b.y)<=p.y&&p.y<=max(a.y,
        b.y);
}
bool segmentsIntersect(Point a,Point b,Point c,Point d){
    long long x=orient(a,b,c),y=orient(a,b,d),z=
        orient(c,d,a),w=orient(c,d,b);
    if(x==0&&onSegment(a,b,c))return
        true;if(y==0&&onSegment(a,b,d))return true;
    if(z==0&&onSegment(c,d,a))return
        true;if(w==0&&onSegment(c,d,b))return true;
```

```
    return (x<0)!=(y<0)&&(z<0)!=(w<0);
}
long long doubledPolygonArea(const vector<Point>& p){
    long long area=0;int n=p.size();
    for(int i=0;i<n;++i)area+=p[i].x*p[(i+1)%n].y-
        p[i].y*p[(i+1)%n].x;
    return llabs(area); // shoelace formula; actual area
        is area/2
}
```

5.3 Convex Hull [14 lines] - ED89B1F4

```
// Monotonic chain hull, O(n log n). Removes collinear
    interior points.
struct Point { long long x,y; bool operator<(const
    Point& o) const { return tie(x,y)<tie(o.x,o.y); }
    };
long long cross(Point a,Point b,Point c) {
    return (b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-a.x);
}
vector<Point> convexHull(vector<Point> p) {
    sort(p.begin(),p.end());
        p.erase(unique(p.begin(),p.end(),[](auto a,auto
        b){return a.x==b.x&&a.y==b.y;}),p.end());
    if (p.size()<=1) return p;
    vector<Point> h;
    for (auto x:p) { while (h.size()>=2 &&
        cross(h[h.size()-2],h.back(),x)<=0)
        h.pop_back(); h.push_back(x); }
    int lower=h.size();
    for (int i=(int)p.size()-2;i>=0;--i) { auto x=p[i];
        while ((int)h.size()>lower &&
        cross(h[h.size()-2],h.back(),x)<=0)
        h.pop_back(); h.push_back(x); }
    h.pop_back(); return h; // counterclockwise; for
        all-collinear input, returns endpoints.
}
```

5.4 Rotating Calipers [18 lines] - 4FFB9253

```
// Diameter (farthest pair) of a convex polygon in CCW
    order, O(n).
// Use the convexHull template first; squared distances
    avoid floating point.
long long dist2(Point a,Point b) {
    long long dx=a.x-b.x, dy=a.y-b.y; return
        dx*dx+dy*dy;
}
long long convexDiameter2(const vector<Point>& p) {
    int n=p.size();
    if (n<=1) return 0;
    if (n==2) return dist2(p[0],p[1]);
    auto area2=[&](int i,int j,int k){ return
        llabs(cross(p[i],p[j],p[k])); };
    long long best=0;
    for (int i=0,j=1;i<n;++i) {
        int ni=(i+1)%n;
        while (area2(i,ni,(j+1)%n)>area2(i,ni,j))
            j=(j+1)%n;
        best=max({best,dist2(p[i],p[j]),dist2(p[ni],
            p[j])});
    }
    return best;
}
```

6 Graph

6.1 2-SAT [33 lines] - 817CF66C

```
// Boolean clauses (x == xv) OR (y == yv). TIME:
    O(V+E).
struct TwoSAT {
    int n;
    vector<vector<int>> g, rg;
    vector<int> order, comp;
    vector<char> seen;
    TwoSAT(int n) : n(n), g(2*n), rg(2*n) {}
    int node(int x, bool value) { return 2*x + !value; }
    void implication(int a, int b) { g[a].push_back(b);
        rg[b].push_back(a); }
    void addOr(int x, bool xv, int y, bool yv) {
        int a = node(x, xv), b = node(y, yv);
        implication(a^1, b); implication(b^1, a);
    }
    void dfs1(int v) {
        seen[v] = 1;
        for (int to : g[v]) if (!seen[to]) dfs1(to);
        order.push_back(v);
    }
    void dfs2(int v, int c) { comp[v] = c; for (int to :
        rg[v]) if (comp[to] == -1) dfs2(to, c); }
    bool solve(vector<int>& value) {
        seen.assign(2*n, 0); order.clear();
        for (int v = 0; v < 2*n; ++v) if (!seen[v])
            dfs1(v);
        comp.assign(2*n, -1);
        for (int i = 2*n-1, c = 0; i >= 0; --i)
            if (comp[order[i]] == -1) dfs2(order[i],
                c++);
        value.resize(n);
        for (int x = 0; x < n; ++x) {
            if (comp[2*x] == comp[2*x+1]) return false;
            value[x] = comp[2*x] > comp[2*x+1];
        }
        return true;
    }
};
```

6.2 Articulation Points and Bridges [26 lines] - 0579D87B

```
// WHEN: find articulation vertices and bridges in an
    undirected graph. TIME: O(V+E). ASSUMES: edge IDs
    distinguish parallel edges.
// Undirected graph; handles parallel edges by tracking
    the parent edge ID.
struct LowLink {
    int n, timer = 0;
    vector<vector<pair<int,int>>> g;
    vector<int> tin, low;
    vector<char> articulation;
    vector<pair<int,int>> bridges;
    LowLink(int n) : n(n), g(n), tin(n, -1), low(n),
        articulation(n) {}
    void addEdge(int u, int v, int id) {
        g[u].push_back({v,id}); g[v].push_back({u,id});
        }
    void dfs(int v, int parentEdge = -1) {
        tin[v] = low[v] = timer++;
        int children = 0;
        for (auto [to,id] : g[v]) {
            if (id == parentEdge) continue;
            if (tin[to] != -1) low[v] = min(low[v],
                tin[to]);
            else {
                dfs(to, id); low[v] = min(low[v],
                    low[to]); ++children;
                if (low[to] > tin[v])
                    bridges.push_back({v,to});
                if (parentEdge != -1 && low[to] >=
                    tin[v]) articulation[v] = true;
            }
        }
        if (parentEdge == -1 && children > 1)
            articulation[v] = true;
    }
    void run() { for (int v = 0; v < n; ++v) if (tin[v]
        == -1) dfs(v); }
};
```

6.3 Bellman-Ford [17 lines] - 682245B2

```
// Single-source shortest paths with negative edges.
    TIME: O(VE).
// Returns false iff a negative cycle is reachable from
    source.
struct Edge { int u, v; long long w; };
bool bellmanFord(int n, int source, const vector<Edge>&
    edges,
                 vector<long long>& dist) {
    const long long INF = (1LL << 60);
    dist.assign(n, INF); dist[source] = 0;
    for (int it = 0; it < n - 1; ++it) {
        bool changed = false;
        for (auto [u, v, w] : edges) if (dist[u] < INF
            && dist[v] > dist[u] + w)
            dist[v] = dist[u] + w, changed = true;
        if (!changed) break;
    }
    for (auto [u, v, w] : edges)
        if (dist[u] < INF && dist[v] > dist[u] + w)
            return false;
    return true;
}
```

6.4 Biconnected Components [36 lines] - 579687C0

```
// Vertex-biconnected edge blocks in an undirected
    graph. TIME: O(V+E).
// Edge IDs handle parallel edges; assumes no
    self-loops.
struct Biconnected {
    int n, timer = 0;
    vector<vector<pair<int,int>>> g;
    vector<int> tin, low, edgeStack;
    vector<char> articulation;
    vector<vector<int>> blocks;
    Biconnected(int n) : n(n), g(n), tin(n, -1),
        low(n), articulation(n) {}
    void addEdge(int u, int v, int id) {
        g[u].push_back({v,id}); g[v].push_back({u,id});
        }
    void dfs(int v, int parentEdge = -1) {
        tin[v] = low[v] = timer++;
        int children = 0;
        for (auto [to, id] : g[v]) {
            if (id == parentEdge) continue;
            if (tin[to] == -1) {
                edgeStack.push_back(id); ++children;
                    dfs(to, id);
                low[v] = min(low[v], low[to]);
                if (low[to] >= tin[v]) {
                    if (parentEdge != -1 || children >
                        1) articulation[v] = 1;
                    vector<int> block;
```

```
                while (!edgeStack.empty()) {
                    int e = edgeStack.back();
                    edgeStack.pop_back();
                    block.push_back(e);
                    if (e == id) break;
                }
                blocks.push_back(move(block));
            }
        } else if (tin[to] < tin[v]) {
            edgeStack.push_back(id);
            low[v] = min(low[v], tin[to]);
        }
    }
}
void build() { for (int v = 0; v < n; ++v) if
    (tin[v] == -1) dfs(v); }
};
```

6.5 Centroid Decomposition [11 lines] - 9BFB863B

```
// WHEN: organize distance queries/updates by tree
    centroids. TIME: O(n log n) decomposition. ASSUMES:
    connected tree; callback handles each chosen
    centroid.
// Centroid decomposition skeleton; callback runs for
    each centroid before recursing.
struct CentroidDecomposition {
    int n; vector<vector<int>> g; vector<int> sub;
        vector<char> blocked;
    CentroidDecomposition(int
        n):n(n),g(n),sub(n),blocked(n){}
    void addEdge(int u,int
        v){g[u].push_back(v);g[v].push_back(u);}
    int calcSize(int v,int p){sub[v]=1;for(int
        u:g[v])if(u!=p&&!blocked[u])sub[v]+=calcSize(u,
        v);return sub[v];}
    int findCentroid(int v,int p,int total){for(int
        u:g[v])if(u!=p&&!blocked[u]&&sub[u]>total
        /2)return findCentroid(u,v,total);return v;}
    function<void(int)> processCentroid=[](int){};
    void build(int entry=0){int
        total=calcSize(entry,-1),c=findCentroid(entry,-
        1,total);processCentroid(c);blocked[c]=1;for(int
        u:g[c])if(!blocked[u])build(u);}
};
```

6.6 DSU on Tree [19 lines] - 17DCA7F1

```
// WHEN: answer per-subtree frequency/statistic queries.
    TIME: O(n log n) with small-to-large traversal.
    ASSUMES: caller supplies addVertex and saveAnswer
    callbacks.
// Sack/DSU-on-tree skeleton. Implement addVertex(v,
    delta) for the statistic you need.
struct DSUOnTree {
    int n; vector<vector<int>> g; vector<int> sub,heavy;
    function<void(int)> saveAnswer; // read the global
        statistic and store it for vertex v
    DSUOnTree(int n):n(n),g(n),sub(n),heavy(n,-1){}
    void addEdge(int u,int
        v){g[u].push_back(v);g[v].push_back(u);}
    function<void(int,int,int)> addVertex; //
        addVertex(v,+1) / removeVertex(v,-1)
    void calcSize(int v,int p){sub[v]=1;int
        best=0;for(int
        u:g[v])if(u!=p){calcSize(u,v);sub[v]+=sub[u];
        if(sub[u]>best)best=sub[u],heavy[v]=u;}}
    void addSubtree(int v,int p,int
        delta){addVertex(v,delta);for(int
        u:g[v])if(u!=p)addSubtree(u,v,delta);}
    void dfs(int v,int p,bool keep){
        for(int
            u:g[v])if(u!=p&&u!=heavy[v])dfs(u,v,false);
        if(heavy[v]!=-1)dfs(heavy[v],v,true);
        for(int u:g[v])if(u!=p&&u!=
            heavy[v])addSubtree(u,v,+1);
        addVertex(v,+1); if(saveAnswer)saveAnswer(v);
        if(!keep)addSubtree(v,p,-1);
    }
    void run(int
        root=0){calcSize(root,-1);dfs(root,-1,true);}
};
```

6.7 Eulerian Path and Circuit [32 lines] - F1583A3D

```
// Undirected multigraph: Hierholzer's algorithm,
    O(V+E).
// Returns a vertex sequence using every edge once, or
    {} if impossible.
vector<int> eulerTrail(int n, const
    vector<pair<int,int>>& edges) {
    if (n == 0) return {};
    int m = edges.size();
    vector<vector<pair<int,int>>> g(n);
    vector<int> degree(n);
    for (int id = 0; id < m; ++id) {
        auto [u, v] = edges[id];
        g[u].push_back({v, id}); g[v].push_back({u,
            id});
        ++degree[u]; ++degree[v];
    }
    vector<int> odd;
    for (int v = 0; v < n; ++v) if (degree[v] & 1)
        odd.push_back(v);
    if (odd.size() != 0 && odd.size() != 2) return {};
    int start = odd.empty() ? 0 : odd[0];
    if (odd.empty()) for (int v = 0; v < n; ++v) if
        (degree[v]) { start = v; break; }
    vector<char> used(m);
    vector<int> it(n), st{start}, path;
    while (!st.empty()) {
        int v = st.back();
        while (it[v] < (int)g[v].size() &&
            used[g[v][it[v]].second]) ++it[v];
        if (it[v] == (int)g[v].size())
            path.push_back(v), st.pop_back();
        else {
            auto [to, id] = g[v][it[v]++];
            if (!used[id]) used[id] = 1,
                st.push_back(to);
        }
    }
    if ((int)path.size() != m + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}
```

6.8 Floyd-Warshall [12 lines] - 3A4CAC57

```
// All-pairs shortest paths. TIME: O(n^3), SPACE:
    O(n^2).
// INF means unreachable. Handles negative edges, but
    not negative cycles.
using ll = long long;
const ll INF = (1LL << 60);
void floydWarshall(vector<vector<ll>>& d) {
    int n = (int)d.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i) if (d[i][k] < INF)
            for (int j = 0; j < n; ++j) if (d[k][j] <
                INF)
                d[i][j] = min(d[i][j], d[i][k] +
                    d[k][j]);
}
// A negative cycle exists if d[u][v] < 0 after the
    algorithm.
```

6.9 Graph Basics [69 lines] - 54305FC5

```
// WHEN: foundational graph traversals/shortest
    paths/MST. TIME: BFS/DFS/toposort O(V+E), Dijkstra
    O((V+E)logV), Kruskal O(E log E). ASSUMES: Dijkstra
    weights nonnegative; 0-1 BFS weights only 0/1.
// Unweighted graph: BFS distances and parents in
    O(V+E).
vector<int> bfsDistances(const vector<vector<int>>& g,
    int source) {
    vector<int> dist(g.size(), -1), parent(g.size(),
        -1);
    queue<int> q; dist[source]=0; q.push(source);
    while(!q.empty()) {
        int v=q.front(); q.pop();
        for(int u:g[v]) if(dist[u]==-1) {
            dist[u]=dist[v]+1; parent[u]=v; q.push(u);
        }
    }
    return dist;
}

// Iterative DFS order; avoids recursion depth issues on
    large graphs.
vector<int> dfsOrder(const vector<vector<int>>& g,int
    source) {
    vector<char> seen(g.size()); vector<int> order,
        st{source}; seen[source]=1;
    while(!st.empty()) {
        int v=st.back(); st.pop_back();
            order.push_back(v);
        for(int u:g[v]) if(!seen[u])
            seen[u]=1,st.push_back(u);
    }
    return order;
}

// Dijkstra: nonnegative edge weights, O((V+E) log V).
vector<long long> dijkstra(const
    vector<vector<pair<int,int>>>& g,int source) {
    const long long INF=(1LL<<62);
    vector<long long> dist(g.size(),INF);
        priority_queue<pair<long
        long,int>,vector<pair<long
        long,int>>,greater<pair<long long,int>>> pq;
    dist[source]=0; pq.push({0,source});
    while(!pq.empty()) {
        auto [d,v]=pq.top(); pq.pop(); if(d!=dist[v])
            continue;
        for(auto [u,w]:g[v]) if(dist[u]>d+w)
            dist[u]=d+w,pq.push({dist[u],u});
    }
    return dist;
}
```

```
// 0-1 BFS: every edge weight must be 0 or 1; O(V+E).
vector<int> zeroOneBFS(const
    vector<vector<pair<int,int>>>& g,int source) {
    const int INF=1e9; vector<int> dist(g.size(),INF);
        deque<int> dq;
    dist[source]=0; dq.push_front(source);
    while(!dq.empty()) {
        int v=dq.front(); dq.pop_front();
        for(auto [u,w]:g[v]) if(dist[u]>dist[v]+w) {
            dist[u]=dist[v]+w;
                if(w==0)dq.push_front(u);else
                dq.push_back(u);
        }
    }
    return dist;
}

// Topological order by Kahn's algorithm; empty result
    means a directed cycle.
vector<int> topologicalSort(const vector<vector<int>>&
    g) {
    vector<int> indeg(g.size()),order;
    for(auto& edges:g) for(int u:edges) ++indeg[u];
    queue<int> q; for(int
        v=0;v<(int)g.size();++v)if(!indeg[v])q.push(v);
    while(!q.empty()) { int
        v=q.front();q.pop();order.push_back(v);for(int
        u:g[v])if(--indeg[u]==0)q.push(u); }
    if(order.size()!=g.size())return {}; return order;
}

// Kruskal MST for an undirected graph. DSU is local to
    keep this snippet standalone.
struct MSTEdge { int u,v; long long w; };
pair<bool,long long> kruskalMST(int n,vector<MSTEdge>
    edges) {
    sort(edges.begin(),edges.end(),[](auto a,auto
        b){return a.w<b.w;});
    vector<int> p(n),sz(n,1); iota(p.begin(),p.end(),0);
    function<int(int)> find=[&](int v){return
        p[v]==v?v:p[v]=find(p[v]);};
    long long total=0; int used=0;
    for(auto e:edges) { int
        a=find(e.u),b=find(e.v);if(a==b)continue;
        if(sz[a]<sz[b])swap(a,b);p[b]=a;sz[a]+=sz[b];
            total+=e.w;++used; }
    return {used==n-1,total}; // false for disconnected
        graph (except n=1)
}
```

6.10 Heavy Light Decomposition [18 lines] - A3AD48C0

```
// WHEN: path queries/updates on a static tree. TIME:
    O(log^2 n) path query. ASSUMES: 0-based connected
    tree; this template supports point assignment and
    path sum.
// HLD for point assignment and path sum; vertices are
    0-indexed.
struct HLD {
    int n,timer=0; vector<vector<int>> g; vector<int>
        parent,depth,heavy,head,pos,sub;
    vector<long long> base,seg;
    HLD(int n):n(n),g(n),parent(n,-1),depth(n),heavy(n,
        -1),head(n),pos(n),sub(n),base(n),seg(4*n){}
    void addEdge(int u,int
        v){g[u].push_back(v);g[v].push_back(u);}
    int dfsSize(int v,int p){parent[v]=p;sub[v]=1;int
        best=0;
        for(int u:g[v])if(u!=p){depth[u]=depth[v]+1;int
            z=dfsSize(u,v);sub[v]+=z;if(z>best)best=z,
            heavy[v]=u;}return sub[v];}
    void decompose(int v,int
        h){head[v]=h;pos[v]=timer++;
        if(heavy[v]!=-1)decompose(heavy[v],h);
        for(int u:g[v])if(u!=parent[v]&&u!=
            heavy[v])decompose(u,u);}
    void build(int
        root=0){dfsSize(root,-1);decompose(root,root);}
    void segSet(int i,long long x,int p,int l,int
        r){if(l==r){seg[p]=x;return;}int
        m=(l+r)/2;if(i<=m)segSet(i,x,p*2,l,m);else
        segSet(i,x,p*2+1,m+1,r);seg[p]=seg[p*2]+seg[p*
        2+1];}
    long long segSum(int ql,int qr,int p,int l,int
        r)const{if(qr<l||r<ql)return
        0;if(ql<=l&&r<=qr)return seg[p];int
        m=(l+r)/2;return segSum(ql,qr,p*2,l,m)+
        segSum(ql,qr,p*2+1,m+1,r);}
    void setValue(int v,long long
        x){segSet(pos[v],x,1,0,n-1);}
    long long pathSum(int a,int b)const{long long
        ans=0;while(head[a]!=head[b]){if(depth[head[a]]
        <depth[head[b]])swap(a,b);ans+=
        segSum(pos[head[a]],pos[a],1,0,n-1);a=
        parent[head[a]];}if(depth[a]>depth[b])swap(a,
        b);return ans+segSum(pos[a],pos[b],1,0,n-1);}
};
```

6.11 Mo on Tree [25 lines] - 813F811E

```
// WHEN: offline path queries with a toggleable vertex
    statistic. TIME: typically O((n+q)*sqrt(n)) toggles.
    ASSUMES: static tree; query conversion receives the
    LCA.
// Offline path queries using Mo's algorithm on the
    doubled Euler tour.
struct TreeMo {
    struct Query { int l,r,extra,id; };
    int n,timer=0; vector<vector<int>> g; vector<int>
        first,last,euler;
    TreeMo(int n):n(n),g(n),first(n),
        last(n){euler.reserve(2*n);}
    void addEdge(int u,int
        v){g[u].push_back(v);g[v].push_back(u);}
    void dfs(int v,int p){first[v]=euler.size();
        euler.push_back(v);for(int
        u:g[v])if(u!=p)dfs(u,v);last[v]=euler.size();
        euler.push_back(v);}
    void build(int root=0){dfs(root,-1);}
    Query makeQuery(int u,int v,int w,int id)const{
        if(first[u]>first[v])swap(u,v);
        if(w==u)return {first[u],first[v],-1,id};
        return {last[u],first[v],w,id};
    }
    template<class Toggle,class GetAnswer>
    vector<long long> solve(vector<Query> qs,Toggle
        toggle,GetAnswer getAnswer)const{
        int block=max(1,(int)sqrt(euler.size()));
        sort(qs.begin(),qs.end(),[&](auto a,auto b){int
            x=a.l/block,y=b.l/block;return
            x==y?((x&1)?a.r>b.r:a.r<b.r):x<y;});
        vector<long long> ans(qs.size()); vector<char>
            active(n); int L=0,R=-1;
        auto flip=[&](int pos){int v=euler[pos];
            active[v]=!active[v];toggle(v,active[v]);};
        for(auto q:qs){while(L>q.l)flip(--L);while(R
            <q.r)flip(++R);while(L<q.l)flip(L++);
            while(R>q.r)flip(R--);
            if(q.extra!=-1)toggle(q.extra,true);
                ans[q.id]=getAnswer();if(q.extra!=-
                1)toggle(q.extra,false);}
        return ans;
    }
};
```

6.12 Multi-source BFS [15 lines] - CF1E4F90

```
// WHEN: shortest unweighted distance from the nearest
    source. TIME: O(V+E).
// Initialize every source at distance 0 before starting
    BFS.
vector<int> multiSourceBFS(const vector<vector<int>>&
    g, vector<int> sources) {
    vector<int> dist(g.size(), -1);
    queue<int> q;
    for (int s : sources) if (dist[s] == -1) dist[s] =
        0, q.push(s);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int to : g[v]) if (dist[to] == -1) {
            dist[to] = dist[v] + 1;
            q.push(to);
        }
    }
    return dist;
}
```

6.13 SCC [43 lines] - 498BB931

```
/*components: number of SCC.
sz: size of each SCC.
comp: component number of each node.
Create reverse graph.
Run find_scc() to find SCC.
Might need to create condensation graph by
    create_condensed().
Think about indeg/outdeg
for multiple test cases- clear
    adj/radj/comp/vis/sz/topo/condensed.*/
vector<int>adj[mx], radj[mx];

int comp[mx], vis[mx], sz[mx], components;
vector<int>topo;
void dfs(int u) {
  vis[u] = 1;
  for (int v : adj[u])
    if (!vis[v]) dfs(v);
  topo.push_back(u);
}
void dfs2(int u, int val) {
  comp[u] = val;
  sz[val]++;
  for (int v : radj[u])
    if (comp[v] == -1)
      dfs2(v, val);
}
```

```
void find_scc(int n) {
  memset(vis, 0, sizeof vis);
  memset(comp, -1, sizeof comp);
  for (int i = 1;i <= n;i++)
    if (!vis[i])
      dfs(i);
  reverse(topo.begin(), topo.end());
  for (int u : topo)
    if (comp[u] == -1)
      dfs2(u, ++components);
}
vector<int>condensed[mx];
void create_condensed(int n) {
  for (int i = 1;i <= n;i++)
    for (int v : adj[i])
      if (comp[i] != comp[v])
        condensed[comp[i]].push_back(comp[v]);
}
```

6.14 Tree DP [18 lines] - 3557D8FE

```
// WHEN: compute subtree sizes and rerooted distance
    sums. TIME: O(n). ASSUMES: connected tree, 0-based
    vertices; recursive DFS may need an iterative
    rewrite for extreme depth.
// Tree DP: subtree sizes/sums, then reroot to get
    distance sum from every vertex.
// O(n), tree vertices 0..n-1; sumDist[u] = sum of
    distances from u to all vertices.
struct RerootTreeDP {
    int n; vector<vector<int>> g; vector<int> sub;
        vector<long long> down,sumDist;
    RerootTreeDP(int
        n):n(n),g(n),sub(n),down(n),sumDist(n){}
    void addEdge(int u,int
        v){g[u].push_back(v);g[v].push_back(u);}
    void dfs1(int v,int p){
        sub[v]=1; down[v]=0;
        for(int u:g[v])if(u!=p){dfs1(u,v);sub[v]+=
            sub[u];down[v]+=down[u]+sub[u];}
    }
    void dfs2(int v,int p){
        for(int u:g[v])if(u!=p){sumDist[u]=sumDist[v]+
            n-2LL*sub[u];dfs2(u,v);}
    }
    vector<long long> run(int root=0){
        dfs1(root,-1);sumDist[root]=down[root];
            dfs2(root,-1);return sumDist;
    }
};
```

6.15 Tree Queries [28 lines] - 738F3996

```
// WHEN: LCA, ancestor checks, and subtree intervals.
    TIME: O(log n) per LCA/lift. ASSUMES: rooted tree
    with vertices 1..n; call dfs(root,0).
// Binary lifting LCA and Euler intervals for one rooted
    tree (vertices 1..n).
struct TreeQueries {
    int n, LOG, timer = 0;
    vector<vector<int>> g, up;
    vector<int> depth, tin, tout, euler, doubleEuler;
    TreeQueries(int n) : n(n), LOG(1), g(n+1),
        depth(n+1), tin(n+1), tout(n+1) {
        while ((1 << LOG) <= n) ++LOG;
        up.assign(LOG, vector<int>(n+1));
            euler.reserve(n);
    }
    void addEdge(int u, int v) { g[u].push_back(v);
        g[v].push_back(u); }
    void dfs(int v, int p) {
        tin[v] = timer++; euler.push_back(v);
            doubleEuler.push_back(v);
        up[0][v] = p;
        for (int j = 1; j < LOG; ++j) up[j][v] =
            up[j-1][up[j-1][v]];
        for (int to : g[v]) if (to != p) { depth[to] =
            depth[v] + 1; dfs(to, v); }
        tout[v] = timer; doubleEuler.push_back(v);
    }
    int lift(int v, int k) const { for (int j = 0; j <
        LOG; ++j) if (k >> j & 1) v = up[j][v]; return
        v; }
    int lca(int a, int b) const {
        if (depth[a] < depth[b]) swap(a, b);
        a = lift(a, depth[a] - depth[b]);
        if (a == b) return a;
        for (int j = LOG - 1; j >= 0; --j) if (up[j][a]
            != up[j][b]) a = up[j][a], b = up[j][b];
        return up[0][a];
    }
    bool isAncestor(int a, int b) const { return tin[a]
        <= tin[b] && tout[b] <= tout[a]; }
};
```

6.16 kuhn [20 lines] - 3D9B43DE

```
// Kuhn's algorithm for maximum bipartite matching; left
    side vertices 0..n-1.
struct Kuhn {
    int n, m;
    vector<vector<int>> g;
    vector<int> matchRight, seen;
    int timer = 0;
    Kuhn(int n, int m) : n(n), m(m), g(n),
        matchRight(m, -1), seen(n) {}
    void addEdge(int left, int right) {
        g[left].push_back(right); }
    bool dfs(int v) {
        if (seen[v] == timer) return false;
        seen[v] = timer;
        for (int to : g[v]) if (matchRight[to] == -1 ||
            dfs(matchRight[to])) { matchRight[to] = v;
            return true; }
        return false;
    }
    int maximumMatching() {
        int ans = 0;
        for (int v = 0; v < n; ++v) { ++timer; ans +=
            dfs(v); }
        return ans;
    }
};
```

7 Math

7.1 CRT [17 lines] - 6647AF89

```
// Merge x=a (mod m), x=b (mod n), including non-coprime
    moduli.
long long extendedGcdCRT(long long a,long long b,long
    long& x,long long& y) {
    if(!b) { x=1; y=0; return a; }
    long long x1,y1,g=extendedGcdCRT(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1; return g;
}
// Returns {residue, modulus}; modulus 0 means no
    solution. Intermediate products must fit.
pair<long long,long long> mergeCRT(long long a,long long
    m,long long b,long long n) {
    long long x,y,g=extendedGcdCRT(m,n,x,y), diff=b-a;
    if(diff%g) return {0,0};
    long long mod=n/g;
    __int128 k=(__int128)(diff/g)*x%mod;
    long long lcm=m*mod;
    long long r=(long
        long)(((__int128)a+(__int128)m*k)%lcm);
    if(r<0) r+=lcm;
    return {r,lcm};
}
```

7.2 Combinatorics and Probability [34 lines] - 103E7315

```
// Formula reference: see the PDF Random section (10)
    for Catalan, Stirling, derangements, and
    number-theory identities.
// Factorials and nCr modulo prime MOD. Requires n <
    MOD.
const long long MOD = 1000000007;
long long modPow(long long a, long long e) {
    long long r = 1;
    for (; e; e >>= 1, a = a * a % MOD) if (e & 1) r = r
        * a % MOD;
    return r;
}
struct Combinatorics {
    vector<long long> fact, invFact;
    Combinatorics(int n) : fact(n + 1), invFact(n + 1) {
        fact[0] = 1;
        for (int i = 1; i <= n; ++i) fact[i] = fact[i-1]
            * i % MOD;
        invFact[n] = modPow(fact[n], MOD - 2);
        for (int i = n; i; --i) invFact[i-1] =
            invFact[i] * i % MOD;
    }
    long long nCr(int n, int r) const {
        if (r < 0 || r > n) return 0;
        return fact[n] * invFact[r] % MOD * invFact[n-r]
            % MOD;
    }
    long long nPr(int n, int r) const { return r < 0 ||
        r > n ? 0 : fact[n] * invFact[n-r] % MOD; }
};

// Stars and bars: ways to distribute n identical items
    into k nonempty groups = C(n-1,k-1).
// Derangements: D[0]=1, D[1]=0,
    D[n]=(n-1)(D[n-1]+D[n-2]).
// Catalan: C[n]=C(2n,n)/(n+1), valid modulo prime when
    factorial range is supported.
// Stirling 2nd kind: S(n,k)=k*S(n-1,k)+S(n-1,k-1),
    S(0,0)=1.
// Code: vector<vector<long long>> S(n+1,vector<long
    long>(k+1)); S[0][0]=1;
// for i=1..n, j=1..min(i,k):
    S[i][j]=(j*S[i-1][j]+S[i-1][j-1]) % MOD.
// PIE: count union = sum(nonempty subsets A)
    (-1)^(|A|+1) count(intersection A).
// Probability: favorable equally-likely outcomes /
    total outcomes.
// Expected value: E[X]=sum_i E[X_i] (linearity, no
    independence needed).
```

```
// Indicator trick: write a count as a sum of 0/1
    indicators, then take expectations.
// Coupon collector for n equally likely types: expected
    draws = n * H_n.
```

7.3 Euler Phi and Mobius [24 lines] - 4089BFF8

```
// Euler phi and Mobius values for every x <= n, linear
    sieve O(n).
vector<int> phiMobiusSieve(int n, vector<int>& mu) {
    vector<int> phi(n+1), lp(n+1), primes;
        mu.assign(n+1,0);
    if(n>=1) phi[1]=mu[1]=1;
    for(int i=2;i<=n;++i) {
        if(!lp[i]) { lp[i]=i; primes.push_back(i);
            phi[i]=i-1; mu[i]=-1; }
        for(int p:primes) {
            if(p>lp[i] || 1LL*i*p>n) break;
            lp[i*p]=p;
            if(i%p==0) { phi[i*p]=phi[i]*p; mu[i*p]=0;
                break; }
            phi[i*p]=phi[i]*(p-1); mu[i*p]=-mu[i];
        }
    }
    return phi;
}
long long eulerPhi(long long n) {
    long long result=n;
    for(long long p=2;p*p<=n;++p) if(n%p==0) {
        while(n%p==0) n/=p;
        result-=result/p;
    }
    if(n>1) result-=result/n;
    return result;
}
```

7.4 Euler Theorem and Power Towers [16 lines] - 06028EF3

```
Euler's theorem: if gcd(a, m) = 1, then a^phi(m) = 1
    (mod m).
Therefore a^b mod m = a^(b mod phi(m)) mod m only when
    gcd(a,m)=1.

For a power tower such as a^(b^c) mod m:
1. If gcd(a,m)=1, reduce the exponent modulo phi(m),
    recursively.
2. If gcd(a,m) != 1, blindly reducing the exponent can
    be wrong.
   Use a proven non-coprime tower method, or split prime
       powers and use CRT.
3. Stop the recursion once the modulus becomes 1. Handle
    0^0 according to
   the problem statement.

Useful totient facts:
- phi(p^k) = p^k - p^(k-1) for prime p.
- If gcd(m,n)=1, phi(m*n) = phi(m)*phi(n).
- phi(n) = n * product over distinct p|n of (1 - 1/p).

This note is a safe reduction guide; it is not a
    complete arbitrary-tower solver.
```

7.5 Exact GCD Counting [16 lines] - F3CAC16C

```
// Count pairs from two arrays with each exact gcd. O(V
    log V), V=max value.
// divisors[d] counts pairs where both values are
    divisible by d.
vector<long long> exactGcdPairs(const vector<int>&
    a,const vector<int>& b,int maxValue) {
    vector<long long> ca(maxValue+1), cb(maxValue+1),
        exact(maxValue+1);
    for (int x:a) ++ca[x];
    for (int x:b) ++cb[x];
    for (int d=maxValue;d>=1;--d) {
        long long count=0;
        for (int x=d;x<=maxValue;x+=d) count+=ca[x];
        long long other=0;
        for (int x=d;x<=maxValue;x+=d) other+=cb[x];
        exact[d]=count*other;
        for (int x=2*d;x<=maxValue;x+=d)
            exact[d]-=exact[x];
    }
    return exact; // exact[d] counts ordered cross-array
        pairs with gcd exactly d.
}
```

7.6 Linear Diophantine Equation [15 lines] - 8747D6A7

```
// Solve a*x+b*y=c. Returns false if no integer solution
    exists.
long long egcd(long long a,long long b,long long& x,long
    long& y) {
    if(!b) { x=1; y=0; return a; }
    long long x1,y1,g=egcd(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1; return g;
}
bool solveDiophantine(long long a,long long b,long long
    c,long long& x,long long& y) {
    if(a==0&&b==0){x=y=0;return c==0;}
    long long g=egcd(llabs(a),llabs(b),x,y);
    if(c%g) return false;
    x*=c/g; y*=c/g;
    if(a<0) x=-x; if(b<0) y=-y;
    return true;
}
// All solutions: x += (b/g)*t, y -= (a/g)*t.
```

7.7 Lucas Theorem [21 lines] - 937ED62C

```
// nCr modulo a prime p, even when n >= p. TIME: O(p +
    log_p n).
// Requires p small enough to precompute factorials up
    to p-1.
long long modPow(long long a,long long e,long long mod)
    {
    long long r=1;
    for (;e;e>>=1,a=a*a%mod) if (e&1) r=r*a%mod;
    return r;
}
long long chooseSmall(long long n,long long r,const
    vector<long long>& fact,long long p) {
    if (r<0 || r>n) return 0;
    return fact[n]*modPow(fact[r]*fact[n-r]%p,p-2,p)%p;
}
long long lucas(long long n,long long r,long long
    p,const vector<long long>& fact) {
    long long ans=1;
    while (n || r) {
        long long ni=n%p, ri=r%p;
        if (ri>ni) return 0;
        ans=ans*chooseSmall(ni,ri,fact,p)%p;
        n/=p; r/=p;
    }
    return ans;
}
```

7.8 Miller-Rabin Pollard-Rho [43 lines] - 0FC59EF1

```
// Deterministic primality test and factorization for
    64-bit unsigned integers.
using u64 = unsigned long long;
using u128 = __uint128_t;
u64 mulMod(u64 a,u64 b,u64 mod) { return (u128)a*b%mod;
    }
u64 powMod64(u64 a,u64 e,u64 mod) {
    u64 r=1;
    for (;e;e>>=1,a=mulMod(a,a,mod)) if (e&1)
        r=mulMod(r,a,mod);
    return r;
}
bool isPrime64(u64 n) {
    if (n<2) return false;
    for (u64 p:{2ULL,3ULL,5ULL,7ULL,11ULL,13ULL,17ULL,
        19ULL,23ULL,29ULL,31ULL,37ULL}) {
        if (n%p==0) return n==p;
    }
    u64 d=n-1, s=0; while (!(d&1)) d>>=1,++s;
    for (u64 a:{2ULL,325ULL,9375ULL,28178ULL,450775ULL,
        9780504ULL,1795265022ULL}) {
        if (a%n==0) continue;
        u64 x=powMod64(a%n,d,n);
        if (x==1 || x==n-1) continue;
        bool composite=true;
        for (int r=1;r<s;++r) { x=mulMod(x,x,n); if
            (x==n-1) { composite=false; break; } }
        if (composite) return false;
    }
    return true;
}
auto seedClock = chrono::steady_clock::now();
mt19937_64 rng64(seedClock.time_since_epoch().count());
u64 pollardRho(u64 n) {
    if (n%2==0) return 2;
    if (n%3==0) return 3;
    while (true) {
        u64 c=rng64()%(n-1)+1, x=rng64()%(n-2)+2, y=x,
            d=1;
        auto f=[&](u64 v){ return
            (u64)(((u128)mulMod(v,v,n)+c)%n); };
        while (d==1) { x=f(x); y=f(f(y));
            d=gcd(x>y?x-y:y-x,n); }
        if (d!=n) return d;
    }
}
void factor64(u64 n,map<u64,int>& factors) {
    if (n==1) return;
    if (isPrime64(n)) { ++factors[n]; return; }
    u64 d=pollardRho(n);
    factor64(d,factors); factor64(n/d,factors);
}
```

7.9 Modular Inverse [13 lines] - 6D6C77FA

```
// Extended Euclid works for any modulus when
    gcd(a,m)=1.
long long extendedGcd(long long a,long long b,long long&
    x,long long& y) {
    if(!b) { x=1; y=0; return a; }
    long long x1,y1,g=extendedGcd(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1; return g;
}
long long modInverse(long long a,long long mod) {
    long long x,y,g=extendedGcd(a,mod,x,y);
    if(g!=1 && g!=-1) return -1; // no inverse
```

```
    x/=g;
    return (x%mod+mod)%mod;
}
// For prime MOD and a%MOD != 0: inverse = a^(MOD-2) mod
    MOD.
```

7.10 Probability and Expected Value [13 lines] - 9669F842

```
// For equally likely outcomes: P(A)=favorable/total;
    conditional P(A|B)=P(A and B)/P(B).
// Linearity: E[X]=sum E[I_i] for indicator variables,
    even when dependent.
// Variance: E[X^2] - E[X]^2; independent variables have
    additive variances.
double probability(int favorable,long long total){return
    (double)favorable/total;}

// Expected DP on an acyclic state graph, evaluated in
    reverse topological order.
// E[u] = immediateCost + sum(probability(u->v) * E[v]);
    terminal states have E=0.
double expectedFromTransitions(double cost,const
    vector<pair<double,double>>& nextStateExpected) {
    double e=cost; for(auto
        [p,nextE]:nextStateExpected)e+=p*nextE; return
        e;
}
// Self-referential expectation example: fair coin until
    HH.
// E0=1+(E0+E1)/2, E1=1+E0/2; solving gives E0=6
    tosses.
// Cyclic expectation states require solving a linear
    system; naive recursion loops.
```

7.11 Stirling Numbers [7 lines] - D3007645

```
// Stirling numbers of the second kind S(n,k) modulo
    MOD.
vector<vector<long long>> stirlingSecondKind(int n,int
    k,long long MOD) {
    vector<vector<long long>> s(n+1,vector<long
        long>(k+1)); s[0][0]=1;
    for(int i=1;i<=n;++i) for(int j=1;j<=min(i,k);++j)
        s[i][j]=(j*s[i-1][j]+s[i-1][j-1])%MOD;
    return s;
}
```

7.12 Sum of Divisors [12 lines] - FCF9A6F7

```
// Sum of positive divisors of n from prime
    factorization: n=product p^e.
long long sumDivisors(long long n) {
    long long answer=1;
    for(long long p=2;p<=n/p;++p) if(n%p==0) {
        long long term=1, power=1;
        while(n%p==0) { n/=p; power*=p; term+=power; }
        answer*=term;
    }
    if(n>1) answer*=1+n;
    return answer;
}
// May overflow long long; use __int128 or modular
    arithmetic for large answers.
```

7.13 XOR Hashing [15 lines] - 5F1277A0

```
// WHEN: randomized fingerprints for set-like data.
    TIME: O(1) per key. ASSUMES: collisions remain
    possible; not cryptographic.
// Randomized XOR fingerprints for sets/multisets of
    small integer keys.
// Collision probability is small, never zero; do not
    use as a cryptographic hash.
uint64_t splitmix64(uint64_t x) {
    x+=0x9e3779b97f4a7c15ULL;
        x=(x^(x>>30))*0xbf58476d1ce4e5b9ULL;
    x=(x^(x>>27))*0x94d049bb133111ebULL; return
        x^(x>>31);
}
struct XorHash {
    uint64_t seed = chrono::steady_clock::now()
        .time_since_epoch()
        .count();
    uint64_t operator()(uint64_t x) const { return
        splitmix64(x+seed); }
};
// Set fingerprint: XOR hash(x) for every element;
    add/remove toggles the same value.
// For multisets, XOR loses even multiplicities; use a
    sum of random hashes instead.
```

8 Misc

8.1 Bit hacks [12 lines] - 1398A0E8

```
# x & -x is the least bit in x.
# iterate over all the subsets of the mask
for (int s=m; ; s=(s-1)&m) {
 ... you can use s ...
 if (s==0)  break;
}
# c = x&-x, r = x+c; (((r^x) >> 2)/c) | r is the
next number after x with the same number of bits set.
# __builtin_popcount(x) //number of ones in binary
  __builtin_popcountll(x) // for long long
# __builtin_clz(x) // number of leading zeros
  __builtin_ctz(x) // number of trailing zeros, they
      also have long long version
```

8.2 STL Reference [28 lines] - 14293C11

```
// Handy STL patterns. Include <bits/stdc++.h> and use
    namespace std.
vector<int> a{1,2,3,4}, b{2,4,6}, v{1,3,5};
set<int> s{1,4,7};
// nth_element: puts the element that would be at k in
    sorted order at position k.
// Average O(n); elements on either side are not fully
    sorted.
// nth_element(a.begin(), a.begin()+k, a.end()); // k-th
    smallest, 0-indexed

// Iterator helpers: distance(first,last),
    advance(it,k), next(it), prev(it).
// For vector iterators these are O(1); for set/map
    iterators they are O(n).
auto it = lower_bound(v.begin(), v.end(), x); // sorted
    vector: first value >= x
auto jt = next(s.begin());                    // ordered
    containers: move by one

// Set algorithms require sorted ranges and write into
    an output iterator.
vector<int> out;
set_intersection(a.begin(),a.end(),b.begin(),b.end(),
    back_inserter(out));
// Also: set_union, set_difference,
    set_symmetric_difference.
// bitset: fixed compile-time number of bits, useful for
    compact state sets.
bitset<1000> mask; mask.set(3); mask.flip(5); bool has
    = mask.test(3);

// tuple: group values of different types. Structured
    binding since C++17.
tuple<int,string,long long> item={1,"name",42};
auto [id,name,value]=item;

// transform: apply a function elementwise. partition:
    split by a predicate.
vector<int> squares(a.size());
transform(a.begin(),a.end(),squares.begin(),[](int
    x){return x*x;});
auto middle=partition(a.begin(),a.end(),[](int x){return
    x%2==0;});
```

8.3 Template [17 lines] - 322F3F47

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T = 1;
    // cin >> T; // uncomment when the input has
        multiple test cases
    while (T--) {
        // solve one test case
    }
    return 0;
}
```

9 Searching

9.1 Basic Array Patterns [38 lines] - 1DE99FBA

```
// 1D prefix sums: inclusive range sum [l,r] in O(1)
    after O(n) build.
vector<long long> buildPrefix(const vector<int>& a) {
    vector<long long> pref(a.size()+1);
    for (int i=0;i<(int)a.size();++i)
        pref[i+1]=pref[i]+a[i];
    return pref;
}
long long rangeSum(const vector<long long>& pref,int
    l,int r) {
    return pref[r+1]-pref[l];
}

// Difference array: add value to every index in
    inclusive [l,r].
// After all updates, prefix-sum diff once to restore
    the final values.
void rangeAdd(vector<long long>& diff,int l,int r,long
    long value) {
    diff[l]+=value;
    if (r+1<(int)diff.size()) diff[r+1]-=value;
}

// Sliding window for nonnegative values: longest
    subarray with sum <= limit.
```

```cpp
int longestWindowAtMost(const vector<int>& a,long long
    limit) {
    int left=0,best=0; long long sum=0;
    for (int right=0;right<(int)a.size();++right) {
        sum+=a[right];
        while (left<=right && sum>limit) sum-=a[left++];
        best=max(best,right-left+1);
    }
    return best;
}

// Two pointers on a sorted array: determine whether two
    values sum to target.
bool hasPairSum(const vector<long long>& a,long long
    target) {
    int l=0,r=(int)a.size()-1;
    while (l<r) {
        long long sum=a[l]+a[r];
        if (sum==target) return true;
        if (sum<target) ++l; else --r;
    }
    return false;
}
```

9.2 Basic Binary Search [27 lines] - 2333BA5B

```cpp
// lower_bound on a sorted vector: first index i with
    a[i] >= x, or n.
int firstAtLeast(const vector<long long>& a,long long x)
    {
    int l=0,r=a.size();
    while (l<r) { int mid=l+(r-l)/2; if (a[mid]<x)
        l=mid+1; else r=mid; }
    return l;
}

// Last true for a monotone predicate: P(l)=true,
    P(r)=false; returns l.
template<class Predicate>
long long lastTrue(long long l,long long r,Predicate P)
    {
    while (r-l>1) {
        long long mid=l+(r-l)/2;
        if (P(mid)) l=mid; else r=mid;
    }
    return l;
}

// Binary search on answer: find the minimum feasible
    value.
// Establish infeasible low and feasible high bounds
    before calling the loop.
template<class Predicate>
long long firstTrue(long long low,long long
    high,Predicate feasible) {
    while (high-low>1) {
        long long mid=low+(high-low)/2;
        if (feasible(mid)) high=mid; else low=mid;
    }
    return high;
}
```

9.3 Range and Search Techniques [41 lines] - 8926C3DB

```cpp
// 2D prefix sum: build then sum inclusive rectangle
    (x1,y1)..(x2,y2).
vector<vector<long long>> prefix2D(const
    vector<vector<int>>& a) {
    int n = a.size(), m = n ? a[0].size() : 0;
    vector<vector<long long>> p(n + 1, vector<long
        long>(m + 1));
    for (int i = 1; i <= n; ++i) for (int j = 1; j <=
        m; ++j)
        p[i][j] = a[i-1][j-1] + p[i-1][j] + p[i][j-1] -
            p[i-1][j-1];
    return p;
}
long long rectangleSum(const vector<vector<long long>>&
    p, int x1, int y1, int x2, int y2) {
    return p[x2+1][y2+1] - p[x1][y2+1] - p[x2+1][y1] +
        p[x1][y1];
}

// Ternary search for a unimodal function on a closed
    interval.
double ternarySearch(double l, double r,
    function<double(double)> f) {
    for (int it = 0; it < 100; ++it) {
        double m1 = l + (r-l)/3, m2 = r - (r-l)/3;
        if (f(m1) < f(m2)) l = m1; else r = m2; //
            maximum; reverse for minimum
    }
    return (l+r)/2;
}

// Meet in the middle: all subset sums, O(2^(n/2)) time
    and memory per half.
vector<long long> subsetSums(const vector<long long>&
    a, int l, int r) {
    vector<long long> sums{0};
    for (int i = l; i < r; ++i) {
        int old = sums.size();
        for (int j = 0; j < old; ++j)
            sums.push_back(sums[j] + a[i]);
    }
    return sums;
}
// 4-sum existence with four distinct indices, expected
    O(n^2) time and memory.
bool fourSum(const vector<long long>& a, long long
    target) {
    unordered_set<long long> earlierPairSums;
    int n=a.size();
    for (int i=0;i<n;++i) {
        for (int j=i+1;j<n;++j)
            if (earlierPairSums.count(target-a[i]-a[j]))
                return true;
        for (int x=0;x<i;++x)
            earlierPairSums.insert(a[x]+a[i]);
    }
    return false;
}
```

10 String

10.1 Aho-Corasick [27 lines] - A686271F

```cpp
// Lowercase a-z patterns; reports number of matches per
    pattern after scanning text.
struct AhoCorasick {
    struct Node { int next[26]; int link = 0;
        vector<int> out; Node() {
        fill(next,next+26,-1); } };
    vector<Node> t{Node()};
    void add(const string& s, int id) {
        int v=0; for(char c:s) { int x=c-'a';
            if(t[v].next[x]==-1) {
            t[v].next[x]=t.size(); t.emplace_back(); }
            v=t[v].next[x]; }
        t[v].out.push_back(id);
    }
    void build() {
        queue<int> q;
        for(int c=0;c<26;++c) { int u=t[0].next[c];
            if(u==-1) t[0].next[c]=0; else q.push(u); }
        while(!q.empty()) {
            int v=q.front(); q.pop();
            for(int id:t[t[v].link].out)
                t[v].out.push_back(id);
            for(int c=0;c<26;++c) {
                int u=t[v].next[c];
                if(u==-1)
                    t[v].next[c]=t[t[v].link].next[c];
                else { t[u].link=t[t[v].link].next[c];
                    q.push(u); }
            }
        }
    }
    vector<int> countMatches(const string& text,int
        patternCount) const {
        vector<int> count(patternCount); int v=0;
        for(char c:text) { v=t[v].next[c-'a']; for(int
            id:t[v].out) ++count[id]; }
        return count;
    }
};
```

10.2 KMP [17 lines] - A1049B24

```cpp
vector<int> prefixFunction(const string& s) {
    vector<int> pi(s.size());
    for(int i=1;i<(int)s.size();++i) {
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) ++j;
        pi[i]=j;
    }
    return pi;
}
vector<int> findMatches(const string& text,const string&
    pattern) {
    if(pattern.empty()) return {};
    string s=pattern+'#'+text; vector<int>
        pi=prefixFunction(s), matches;
    for(int i=(int)pattern.size()+1;i<(int)s.size();++i)
        if(pi[i]==(int)pattern.size())
            matches.push_back(i-2*(int)pattern.size());
    return matches;
}
```

10.3 Manacher [15 lines] - FBB15D9E

```cpp
// Longest palindromic substring length in O(n).
int longestPalindrome(const string& s) {
    string t="^";
    for(char c:s) t += string("#") + c;
    t += "#$";
    vector<int> p(t.size()); int
        center=0,right=0,best=0;
    for(int i=1;i+1<(int)t.size();++i) {
        int mirror=2*center-i;
        if(i<right) p[i]=min(right-i,p[mirror]);
        while(t[i+1+p[i]]==t[i-1-p[i]]) ++p[i];
```

```
        if(i+p[i]>right) center=i,right=i+p[i];
        best=max(best,p[i]);
    }
    return best;
}
```

10.4 Rolling Hash [33 lines] - 87D05FA9

```
// WHEN: compare substrings quickly. TIME: O(n) build
    and O(1) query. ASSUMES: hashes are probabilistic;
    double hash greatly lowers but does not remove
    collision risk.
// One modular polynomial hash for substring equality;
    use two moduli when collisions matter.
struct RollingHash {
    static const long long MOD = 1000000007, BASE =
        911382323;
    vector<long long> h, power;
    RollingHash(const string& s) : h(s.size()+1),
        power(s.size()+1,1) {
        for(int i=0;i<(int)s.size();++i) {
            power[i+1]=power[i]*BASE%MOD;
            h[i+1]=(h[i]*BASE+(unsigned
                char)s[i]+1)%MOD;
        }
    }
    long long get(int l,int r) const { // [l,r)
        long long x=(h[r]-h[l]*power[r-l])%MOD;
        return (x+MOD)%MOD;
    }
};

// Double polynomial hash for substring equality; query
    ranges are [l,r).
struct DoubleRollingHash {
    static const long long MOD1=1000000007,
        MOD2=1000000009, BASE=911382323;
    vector<long long> h1,h2,p1,p2;
    DoubleRollingHash(const string&
        s):h1(s.size()+1),h2(s.size()+1),p1(s.size()+1,
        1),p2(s.size()+1,1){
        for(int i=0;i<(int)s.size();++i){
            long long x=(unsigned char)s[i]+1;
            p1[i+1]=p1[i]*BASE%MOD1;p2[i+1]=p2[i]*
                BASE%MOD2;
            h1[i+1]=(h1[i]*BASE+x)%MOD1;h2[i+1]=(h2[i]*
                BASE+x)%MOD2;
        }
    }
    pair<long long,long long> get(int l,int r)const{
        long long a=(h1[r]-h1[l]*p1[r-l])%MOD1,b=
            (h2[r]-h2[l]*p2[r-l])%MOD2;
        if(a<0)a+=MOD1;if(b<0)b+=MOD2;return {a,b};
    }
};
```

10.5 Suffix Array [29 lines] - 7C2108AD

```
// Suffix array + Kasai LCP; O(n log^2 n) build, O(n)
    LCP.
struct SuffixArray {
    vector<int> sa, rank, lcp;
    explicit SuffixArray(const string& s) {
        int n=s.size(); sa.resize(n); rank.resize(n);
            iota(sa.begin(),sa.end(),0);
        for(int i=0;i<n;++i) rank[i]=(unsigned
            char)s[i];
        vector<int> nextRank(n);
        for(int k=1;k<n;k*=2) {
            sort(sa.begin(),sa.end(),[&](int a,int b) {
                if(rank[a]!=rank[b]) return
                    rank[a]<rank[b];
                return (a+k<n?rank[a+k]:-1)<(b+k
                    <n?rank[b+k]:-1);
            });
            if(n) nextRank[sa[0]]=0;
            for(int i=1;i<n;++i) {
                int a=sa[i-1],b=sa[i];
                nextRank[b]=nextRank[a]+(rank[a]!=
                    rank[b] ||
                    (a+k<n?rank[a+k]:-1)!=(b+k
                    <n?rank[b+k]:-1));
            }
            rank.swap(nextRank);
            if(n && rank[sa[n-1]]==n-1) break;
        }
        vector<int> pos(n); for(int i=0;i<n;++i)
            pos[sa[i]]=i;
        lcp.assign(max(0,n-1),0); int h=0;
        for(int i=0;i<n;++i) {
            int r=pos[i]; if(r==n-1) { h=0; continue; }
            int j=sa[r+1]; while(i+h<n && j+h<n &&
                s[i+h]==s[j+h]) ++h;
            lcp[r]=h; if(h) --h;
        }
    }
};
```

10.6 Trie [15 lines] - 48982540

```
// Lowercase a-z trie.
struct Trie {
    struct Node { int next[26]; bool terminal=false;
        Node() { fill(next,next+26,-1); } };
    vector<Node> t{Node()};
    void insert(const string& s) {
        int v=0;
        for(char c:s) { int x=c-'a';
            if(t[v].next[x]==-1) {
            t[v].next[x]=t.size(); t.emplace_back(); }
            v=t[v].next[x]; }
        t[v].terminal=true;
    }
    bool contains(const string& s) const {
        int v=0;
        for(char c:s) { int x=c-'a';
            if(t[v].next[x]==-1) return false;
            v=t[v].next[x]; }
        return t[v].terminal;
    }
};
```

10.7 Z-Algorithm [10 lines] - 7FEC0BF7

```
vector<int> zFunction(const string& s) {
    int n=s.size(); vector<int> z(n); int l=0,r=0;
    for(int i=1;i<n;++i) {
        if(i<r) z[i]=min(r-i,z[i-l]);
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) ++z[i];
        if(i+z[i]>r) l=i,r=i+z[i];
    }
    if(n) z[0]=n;
    return z;
}
```

11 Basic Math and Formula Sheet

11.1 Integer Arithmetic

- Euclidean algorithm: $\gcd(a,b) = \gcd(b, a \bmod b)$; $\operatorname{lcm}(a,b) = |ab|/\gcd(a,b)$ (divide first to reduce overflow).
- Divisibility: if $d \mid a$ and $d \mid b$, then $d \mid (xa+yb)$ for integers x, y.
- For $n > 0$, the number of divisors is $\prod(e_i+1)$ and their sum is $\prod(1+p_i+\cdots+p_i^{e_i})$ when $n = \prod p_i^{e_i}$.

11.2 Sums and Counting

- $1+2+\cdots+n = n(n+1)/2$; $\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6$; $\sum_{i=1}^{n} i^3 = [n(n+1)/2]^2$.
- Arithmetic progression: $S_n = n(a_1+a_n)/2$. Geometric progression: $1+r+\cdots+r^n = (r^{n+1}-1)/(r-1)$ for $r \neq 1$.
- $\binom{n}{k} = \binom{n}{n-k}$ and $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.
- Pigeonhole principle: placing more than k objects into k boxes forces some box to contain at least two objects.

11.3 Modular Arithmetic

- Normalize possibly negative x by $(x\%m+m)\%m$ for $m > 0$.
- $(a+b) \bmod m$, $(a-b) \bmod m$, and $(ab) \bmod m$ can be computed from residues; use a wide integer type for products.
- Division modulo m means multiplication by an inverse. a^{-1} exists exactly when $\gcd(a,m) = 1$.
- For prime p and $p \nmid a$, Fermat gives $a^{p-1} \equiv 1 \pmod{p}$ and $a^{-1} \equiv a^{p-2} \pmod{p}$.

11.4 Binary Search Checklist

- Binary search needs a monotone predicate: once true (or false), it stays that way on the remaining side.
- Maintain a clear invariant such as $P(l) =$ false and $P(r) =$ true; stop when $r - l = 1$.
- Use $l+(r-l)/2$ to avoid midpoint overflow. For answer search, first prove a feasible bound and test feasibility in each step.

11.5 Bit and Geometry Reminders

- $x\&(-x)$ isolates the lowest set bit; $x\&(x-1)$ removes it; a positive power of two satisfies $(x\&(x-1))=0$.
- Squared Euclidean distance is $(x_1-x_2)^2+(y_1-y_2)^2$; compare it instead of taking square roots.
- For integer points, the cross product gives orientation; polygon area is half the absolute shoelace sum. See Basic Geometry.cpp.

11.6 Combinatorics

- $\sum_{k=0}^{n}\binom{n-k}{k}=Fib_{n+1}$
- $\binom{n}{k}+\binom{n}{k+1}=\binom{n+1}{k+1}$
- $k\binom{n}{k}=n\binom{n-1}{k-1}$
- Number of binary sequences of length n such that no two 0's are adjacent $=Fib_{n+1}$
- Number of non-negative solution of $x_1+x_2+x_3+...+x_k=n$ is $\binom{n+k-1}{n}$

11.6.1 Catalan Number

- $C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}=\frac{(2n)!}{(n+1)!n!}$
- $C_0=1, C_1=1, C_n=\sum_{k=0}^{n-1}C_kC_{n-1-k}$
- $1,1,2,5,14,42,132,429,1430,4862,16796,58786$
- Number of correct bracket sequences consisting of n opening brackets.
- Number of ways to completely parenthesize n+1 factors.
- The number of triangulations of a convex polygon with +2 sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).
- The number of ways to connect the 2n points on a circle to form n disjoint i.e. non-intersecting chords.
- The number of monotonic lattice paths from point (0,0) to point (n,n) in a square lattice of size $n\times n$, which do not pass above the main diagonal
- Number of permutation of length n that can be stack sorted.
- The number of non-crossing partitions of a set of n elements.
- The number of rooted full binary tree with n+1 leaves.
- The number of Dyck words of length 2n. A string consisting of n X's and n Y's such that no string prefix has more Y's than X's.
- Number of permutation of length n with no three-term increasing subsequence.
- Number of ways to tile a stairstep shape of height n with n rectangle.
- $C_n^k=\frac{k+1}{n+1}\binom{2n-k}{n-k}$ denote the number of bracket sequences of size 2n with the first k elements being (.
- $N(n,k)=\frac{1}{n}\binom{n}{k}\binom{n}{k-1}$
- The number of expressions containing n pairs of correct parentheses, which contain k distinct nestings. $N(4,2)=6$
  $()((())), (())(()), (()(())), ((()())), ((())()), ((()))()$
- The number of paths from (0,0) to (2n, 0) with steps only northeast and southeast, not staying below the x-axis with k peaks. And sum of all number of peaks is Catalan number.

11.6.2 Stirling Numbers of the Second Kind

- Number of ways to partition a set of n objects into k non-empty subsets.
- $S(n,k)=k*S(n-1,k)+S(n-1,k-1), S(0,0)=1, S(n,0)=S(0,n)=0$
- $S(n,2)=2^{n-1}-1$
- $S(n,k)=\frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$
- $S(n,k)*k!=$ number of ways to color n nodes using colors from 1 to k such that each color is used at least once.

11.6.3 Lucas Theorem

- If p is prime then $\binom{p^a}{k}\equiv 0 \mod p$
- For non-negative integers m and n and a prime p:
  $\binom{m}{n}=\prod_{i=0}^{k}\binom{m_i}{n_i}(\mod p)$ where
  $m=m_kp^k+m_{k-1}p^{k-1}+...+m_1p+m_0$ $n=n_kp^k+n_{k-1}p^{k-1}+...+n_1p+n_0$ are the base p expansion.

11.6.4 Derangement

- A permutation such that no element appears in its original position.
- $d(n)=(n-1)*(d(n-1)+d(n-2)), d(0)=1, d(1)=0$
- $d(n)=nd(n-1)+(-1)^n=\lfloor\frac{n!}{e}\rfloor, n\geq 1$

11.7 Number Theory

11.7.1 Mobius Function and Inversion

- define $\mu(n)$ as the sum of the primitive nth roots of unity depending on the factorization of n into prime factors:

$$\mu(x)=\begin{cases}0 & \text{n is not square free}\\ 1 & \text{n has even number of prime factors}\\ -1 & \text{n has odd number of prime factors}\end{cases}$$

- Mobius Inversion:

$$g(n)=\sum_{d|n}f(d)\leftrightarrow f(n)=\sum_{d|n}\mu(d)g(n/d)$$

- $\sum_{d|n}\mu(d)=[n=1]$
- $\phi(n)=\sum_{d|n}\mu(d).\frac{n}{d}=n\sum_{d|n}\frac{\mu(d)}{d}=\sum_{d|n}d.\mu(\frac{n}{d})$
- $a|b\rightarrow\phi(a)|\phi(b)$
- $\phi(mn)=\phi(m).\phi(n).\frac{d}{\phi(d)}$ where $d=gcd(m,n)$
- $\phi(n^m)=n^{m-1}\phi(n)$
- $\sum_{i=1}^{n}[gcd(i,n)=k]=\phi(\frac{n}{k})$
- $\sum_{i=1}^{n}gcd(i,n)=\sum_{d|n}d.\phi(\frac{n}{d})$
- $\sum_{i=1}^{n}\frac{1}{gcd(i,n)}=\sum_{d|n}\frac{1}{d}.\phi(\frac{n}{d})=\frac{1}{n}\sum_{d|n}d.\phi(d)$
- $\sum_{i=1}^{n}\frac{i}{gcd(i,n)}=\frac{n}{2}.\sum_{d|n}\frac{1}{d}.\phi(\frac{n}{d})=\frac{n}{2}.\frac{1}{n}\sum_{d|n}d.\phi(d)$
- $\sum_{i=1}^{n}\frac{n}{gcd(i,n)}=2.\sum_{i=1}^{n}\frac{i}{gcd(i,n)}-1$

11.7.2 GCD and LCM

- gcd(a,b) = gcd(b, a mod b)
- If $a|b.c$, and gcd(a,b) = d, then $(a/d)|c$.
- GCD is a multiplicative function.
- gcd(a, lcm(b,c)) = lcm(gcd(a,b), gcd(a,c))
- $gcd(n^a-1,n^b-1)=n^{gcd(a,b)}-1$

11.7.3 Pick's Theorem

According to Pick's Theorem We can calculate the area of any polygon by just counting the number of Interior and Boundary lattice points of that polygon. If number of interior points are I and number of boundary lattice points are B then Area (A) of polygon will be:

$$Area = I + B/2 - 1$$

where I is the number of points in the interior shape, B stands for the number of points on the boundary of the shape.

11.7.4 Formula Cheatsheet

- $\sum_{i=1}^{n} = \frac{1}{m+1}[(n+1)^{m+1} - 1 - \sum_{i=1}^{n}((i+1)^{m+1} - i^{m+1} - (m+1)i^m)]$
- $\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}, c \neq 1$
- $\sum_{i=0}^{\infty} c^i = \frac{1}{1-c}, \sum_{i=1}^{\infty} c^i = \frac{c}{1-c}, |c| < 1$
- $H_n = \sum_{i=1}^{n} \frac{1}{n}, \sum_{i=1}^{n} iH_i = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}$
- $\sum_{k=0}^{n} \binom{r+k}{k} = \binom{r+n+1}{n}$